GOOGLE | ALL SOURCES | TOPIC 6 OF 10 | COMMON TOPIC

Strings, Parsing and Recursion

20 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	13	4
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Strings, Parsing and Recursion | 20 questions | tap a title to jump to it

STRINGS (13)

E 1. Given an array of strings, return the words that can be typed using letters of the alphabet on only one r... **Exponent no date**

E 2. Return all unique substrings of a given string. **Exponent no date**

M 3. Snake Case to Camel Case with Repository-Wide Refactor **1Point3Acres 2026-07-14**

M 4. Check Whether Digits Are Monotone Increasing **PracHub 2026-07-04**

M 5. Can board states be transformed? **PracHub 2026-04-01**

M 6. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) **1Point3Acres 2026-02-01**

M 7. Decide string transform with directional tokens and walls **PracHub 2025-07-15**

M 8. Given an array of strings, return groups of "buddy strings," where the strings have the same length and... **Exponent no date**

M 9. Write functions to serialize and deserialize a list of strings. **Exponent no date**

M 10. Determine if two sentences are similar. **Exponent no date**

H 11. Find the Longest Straight Run of One Color **PracHub 2026-08-23**

H 12. Convert Valid Snake Case to Lower Camel Case **PracHub 2026-04-16**

H 13. Find Dictionary Matches for a Jammed String **PracHub 2026-04-09**

RECURSION AND BACKTRACKING (6)

M 14. Mahjong Winning Hand Detection (14 Tiles) **1Point3Acres 2026-10-09**

M 15. Count Prefix-Matching Simple Paths in a Labeled Graph **PracHub 2026-08-09**

M 16. Permutations **Exponent no date**

M 17. Generate Parentheses **Exponent no date**

M 18. Explain how to find a target sum in an array. **Exponent no date**

H 19. Matrix Flower Placement with House Adjacency **1Point3Acres 2026-05-31**

PARSING (1)

E 20. Implement string formatting and request tracking tasks **PracHub 2026-02-11**

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

EASY | Exponent | 1 of 20

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 160/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 500 Keyboard Row (inferred from short statement)

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/7293/return-words-one-keyboard-row

QUESTION

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

APPROACH

Map each letter to its keyboard row; keep words whose letters all belong to one row. O(total chars).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask about case (treat as case-insensitive), empty strings, and non-letter characters.

BRUTE FORCE TO SAY OUT LOUD

For each word test all three rows with string containment. Fine and short.

OPTIMAL IDEA

Map each letter to its keyboard row once (26 entries). A word qualifies if the set of rows of its lowercased letters has one element. O(total characters).

DRY-RUN THESE CASES

Mixed case such as Alaska, a word using two rows, an empty word, a one-letter word.

LIKELY FOLLOW-UPS

Different keyboard layouts as a parameter.

PITFALLS

Forgetting to lowercase before looking up rows.

HOW TO THINK ABOUT IT

Each letter belongs to exactly one row, so the question for a word is whether all its letters map to the same row. Build the letter-to-row lookup once and check each word with a set of rows.

Basis: LeetCode 500, Keyboard Row.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
ROWS = [set("qwertyuiop"), set("asdfghjkl"),
        set("zxcvbnm")]
def brute(words):
    found = []
    for word in words:
        letters = set(word.lower())
        for row in ROWS:
            if letters <= row:
                found.append(word)
                break
    return found
```

TIME	SPACE
O(W * L * 3)	O(1)

WHY IT IS SLOW

Up to three scans per word; building one letter-to-row table makes each word a single scan, O(total characters).

OPTIMAL SOLUTION · Map each letter to its row and compare sets

TIME	SPACE
O(total characters)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the keyboard layout is a parameter (Dvorak, AZERTY, a phone keypad)?

Nothing changes except the row table, which is built from the layout argument. Build the letter-to-row map once and reuse it across queries.

FOLLOW-UP 2 Other layouts (Dvorak, AZERTY) or a phone keypad.

Pass the rows as a parameter and build the letter-to-row dict from it. Nothing else changes.

FOLLOW-UP 3 A different keyboard layout.

Pass the rows as a parameter and build the letter-to-row map from it (a phone keypad or Dvorak works the same way). Letters that are on no row make the word invalid, which the code checks.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Given an array of strings, return the
# words that can be typed using letters of the alphabet on
# only one row of American keyboard.
#
# Given an array of strings, return the words that can be
# typed using letters of the alphabet on only one row of
# American keyboard.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the keyboard layout is a parameter (Dvorak,
# AZERTY, a phone keypad)?
# 2. Other layouts (Dvorak, AZERTY) or a phone keypad.
# 3. A different keyboard layout.
#
# INTERVIEW NOTES
# Each letter belongs to exactly one row, so the question
# for a word is whether all its letters map to the same
# row. Build the letter-to-row lookup once and check each
# word with a set of rows.
#
# BRUTE FORCE (say it first): For each word test whether
# it fits each row
# For every word, check whether all of its lowercase
# letters are in the first row, or all in the second, or
# all in the third. Correct; it rescans each word up to
# three times. Time O(W * L * 3). Space O(1).


# BRUTE FORCE (say it first): For each word test whether
# it fits each row | time O(W * L * 3)
def keyboard_row_brute(words):
    rows = [
        set("qwertyuiop"),
        set("asdfghjkl"),
        set("zxcvbnm"),
    ]
    return [
        w
        for w in words
        if any(set(w.lower()) <= row for row in rows)
    ]


# HELPERS for the base question
QWERTY = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]
```

```
# LAYOUT table (change it for other keyboards)
def row_map(rows):
    # letter -> index of its row; built once and reused
    return {ch: i for i, row in enumerate(rows)
            for ch in row}


# BASE QUESTION (optimal): Map each letter to its row and
# compare sets
```

```
# Idea: Build a dict from letter to row index (26
# entries). A word qualifies if all of its lowercased
# letters map to the same row, i.e. the set of rows has
# size one. Lowercase first so 'Alaska' works.
# FOLLOW-UP 3: A different keyboard layout.
# Answer: Pass the rows as a parameter and build the
# letter-to-row map from it (a phone keypad or Dvorak
# works the same way). Letters that are on no row make the
# word invalid, which the code checks.
def one_row_words(words, rows=QWERTY):
    # FOLLOW-UP 3: pass rows= for another keyboard layout
    row_of = row_map(rows)
    out = []
    for w in words:
        letters = w.lower()
        # a letter that is on no row makes the word
        # invalid
        if any(c not in row_of for c in letters):
            continue
        # all letters must come from the same row
        if len({row_of[c] for c in letters}) <= 1:
            out.append(w)
    return out


if __name__ == "__main__":
    words = ["Hello", "Alaska", "Dad", "Peace"]
    assert one_row_words(words) == ["Alaska", "Dad"]
    assert one_row_words([]) == []
    assert one_row_words(["", "a1"]) == [""]
    # another layout: a phone keypad
    keypad = ["abc", "def", "ghi"]
    got = one_row_words(["bad", "fed", "cab"], keypad)
    assert got == ["fed", "cab"]
```

```
    # Dvorak top row
    dvorak = ["pyfgcrl", "aoeuidhtns", "qjkxbmwvz"]
    assert one_row_words(["tent"], dvorak) == ["tent"]
```

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

EASY | Exponent | 2 of 20

Return all unique substrings of a given string.

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 219/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (substring enumeration) (inferred from short statement)

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/1223/unique-substrings-given-string

QUESTION

Return all unique substrings of a given string.

APPROACH

Double loop adding s[i:j] to a set, O(n^2) substrings (O(n^3) chars). Mention a suffix automaton/trie for large n.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether to return the substrings or just count them, whether the empty string counts, the maximum length of the string, and whether order matters.

BRUTE FORCE TO SAY OUT LOUD

Double loop adding s[i:j] to a set. O(n^2) substrings and up to O(n^3) characters in total.

OPTIMAL IDEA

For small n the set is fine. To count without storing, build a suffix automaton or a suffix array with LCP: the number of distinct substrings is the sum of (n - suffix_start) minus the LCP values, O(n log n). A trie over all suffixes also works at O(n^2).

DRY-RUN THESE CASES

Empty string, a single character, all equal characters (n substrings), a string with no repeats (n(n + 1) / 2).

LIKELY FOLLOW-UPS

Count only, the k-th lexicographic substring, or the longest repeated substring.

PITFALLS

Counting the empty string by accident. Using a list instead of a set and returning duplicates.

HOW TO THINK ABOUT IT

If the output must list the substrings, its size already is quadratic, so a set over all slices is the honest answer. If only the count is needed, nothing needs to be stored: distinct substrings equal total suffix lengths minus the sum of longest-common-prefixes between adjacent sorted suffixes.

Basis: Written; counting distinct substrings is a classic (suffix array with LCP, suffix automaton).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    result = set()
    for i in range(len(s)):
        for j in range(i + 1, len(s) + 1):
            result.add(s[i:j])
    return result
```

TIME	SPACE
O(n^3)	O(n^3)

WHY IT IS SLOW

There are about n^2 / 2 substrings and each slice copies up to n characters: O(n^3) time and space. If only the count is needed, a suffix array with LCP counts them in O(n log n) without storing any.

OPTIMAL SOLUTION · Count with a suffix array and LCP

TIME	SPACE
O(n^2 log n) with naive sorting (O(n log n) with a real suffix array)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List them without building duplicates first.

Insert every suffix into a trie; a substring is new exactly when its trie node is created. O(n^2) time and nodes, and the substrings appear without duplicates.

FOLLOW-UP 2 Return the k-th lexicographic distinct substring, or the longest repeated substring.

Both use the suffix array with LCP: the longest repeated substring is the maximum LCP value, and the k-th substring is found by walking the sorted suffixes and subtracting the new prefixes each one contributes.

FOLLOW-UP 3 The string has a million characters.

Quadratic is out. Build a suffix automaton (O(n)) and count distinct substrings as the sum of len(v) - len(link(v)) over states, or use a suffix array with O(n log n) construction.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Return all unique substrings of a given
# string.
#
# Return all unique substrings of a given string.
#
# FOLLOW-UPS (answered below, in this order):
# 1. List them without building duplicates first.
# 2. Return the k-th lexicographic distinct substring, or
# the longest repeated substring.
# 3. The string has a million characters.
#
# INTERVIEW NOTES
# If the output must list the substrings, its size already
# is quadratic, so a set over all slices is the honest
# answer. If only the count is needed, nothing needs to be
# stored: distinct substrings equal total suffix lengths
# minus the sum of longest-common-prefixes between
# adjacent sorted suffixes.
#
# BRUTE FORCE (say it first): Set of every slice
# Add s[i:j] for every i < j to a set. There are n(n + 1)
# / 2 slices, each up to n characters, so the work and
# memory are cubic in characters. Fine for short strings.
# Time O(n^3). Space O(n^3).


# BRUTE FORCE (say it first): Set of every slice | time
# O(n^3)
def unique_substrings_brute(s):
    return {
        s[i:j]
        for i in range(len(s))
        for j in range(i + 1, len(s) + 1)
    }


# HELPERS for follow-up 2
def suffix_array(s):
    # start positions of all suffixes in sorted order
    return sorted(range(len(s)), key=lambda i: s[i:])


def lcp_array(s, sa):
    # Kasai: lcp[i] = common prefix of suffix sa[i] and
    # the
```

```
    # one before it, found in O(n) in total
    n = len(s)
    rank = [0] * n
    for i, start in enumerate(sa):
        rank[start] = i
    lcp, h = [0] * n, 0
    for start in range(n):
        if rank[start] > 0:
            prev = sa[rank[start] - 1]
            while (start + h < n and prev + h < n
                   and s[start + h] == s[prev + h]):
                h += 1
            lcp[rank[start]] = h
            h = max(h - 1, 0)
        else:
            h = 0
    return lcp


# BASE QUESTION (optimal): Count with a suffix array and
# LCP
# Idea: Sort the suffixes; the number of distinct
# substrings is sum over suffixes of (length) minus sum of
# the LCP of neighbours in sorted order, because each new
# suffix adds only the prefixes that the previous one did
# not already contribute.
def unique_substrings(s):
    # simplest: every slice into a set, then sort. O(n^3).
    return sorted({s[i:j] for i in range(len(s))
                   for j in range(i + 1, len(s) + 1)})


# FOLLOW-UP 1: List them without building duplicates
# first.
# Answer: Insert every suffix into a trie; a substring is
# new exactly when its trie node is created. O(n^2) time
# and nodes, and the substrings appear without duplicates.
def trie_substrings(s):
    # follow-up: no duplicates built first. Insert every
    # suffix into a trie; a substring is new exactly when
    # its
    # node is created while inserting.
    root = {}
    for start in range(len(s)):
        node = root
```

```
        for end in range(start, len(s)):
            ch = s[end]
            if ch not in node:
                node[ch] = {}
                yield s[start:end + 1]
            node = node[ch]


# FOLLOW-UP 2: Return the k-th lexicographic distinct
# substring, or the longest repeated substring.
# Answer: Both use the suffix array with LCP: the longest
# repeated substring is the maximum LCP value, and the
# k-th substring is found by walking the sorted suffixes
# and subtracting the new prefixes each one contributes.
def kth_substring(s, k):
    # follow-up: the k-th distinct substring in dictionary
    # order. Walk the sorted suffixes: each one brings
    # n - start - lcp new prefixes.
    sa = suffix_array(s)
    lcp = lcp_array(s, sa)
    for i, start in enumerate(sa):
        fresh = len(s) - start - lcp[i]
        if k <= fresh:
            return s[start:start + lcp[i] + k]
        k -= fresh
    return None


# FOLLOW-UP 2: Return the k-th lexicographic distinct
# substring, or the longest repeated substring.
# Answer: Both use the suffix array with LCP: the longest
# repeated substring is the maximum LCP value, and the
# k-th substring is found by walking the sorted suffixes
# and subtracting the new prefixes each one contributes.
def longest_repeated(s):
    # follow-up: the biggest lcp value is the longest
    # substring that occurs at least twice
    sa = suffix_array(s)
    lcp = lcp_array(s, sa)
```

```
    if not s or max(lcp) == 0:
        return ""
    best = max(range(len(s)), key=lcp.__getitem__)
    return s[sa[best]:sa[best] + lcp[best]]
```

```
# FOLLOW-UP 3: The string has a million characters.
# Answer: Quadratic is out. Build a suffix automaton
# (O(n)) and count distinct substrings as the sum of
# len(v) - len(link(v)) over states, or use a suffix array
# with O(n log n) construction.
def count_distinct(s):
    # follow-up: a million characters. A suffix automaton
    # is
    # built in O(n); the distinct substrings number
    # sum(len[v] - len[link[v]]) over its states.
    length, link, nxt = [0], [-1], [{}]
    last = 0
    for ch in s:
        cur = len(length)
        length.append(length[last] + 1)
        link.append(0)
        nxt.append({})
        p = last
        while p != -1 and ch not in nxt[p]:
            nxt[p][ch] = cur
            p = link[p]
        if p != -1:
            q = nxt[p][ch]
            if length[p] + 1 == length[q]:
                link[cur] = q
            else:
                # split q: a clone with the shorter length
                clone = len(length)
                length.append(length[p] + 1)
                link.append(link[q])
                nxt.append(dict(nxt[q]))
                while p != -1 and nxt[p].get(ch) == q:
                    nxt[p][ch] = clone
                    p = link[p]
                link[q] = link[cur] = clone
        last = cur
    return sum(length[v] - length[link[v]]
               for v in range(1, len(length)))


if __name__ == "__main__":
    assert unique_substrings("abc") == [
        "a", "ab", "abc", "b", "bc", "c"]
    assert unique_substrings("aaa") == ["a", "aa", "aaa"]
    assert unique_substrings("") == []
```

```
    three = sorted(trie_substrings("aaa"))
    assert three == ["a", "aa", "aaa"]
    assert kth_substring("abc", 4) == "b"
    assert kth_substring("abc", 7) is None
    assert longest_repeated("banana") == "ana"
    assert longest_repeated("abcd") == ""
    assert count_distinct("aaa") == 3
    assert count_distinct("") == 0
    import random
    for _ in range(300):
        s = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 9)))
        want = unique_substrings(s)
        assert sorted(trie_substrings(s)) == want
        assert len(list(trie_substrings(s))) == len(want)
        assert count_distinct(s) == len(want)
        for k, sub in enumerate(want, 1):
            assert kth_substring(s, k) == sub
```

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

Site tags: string, refactoring, medium

MEDIUM | 1Point3Acres | tech-screen | 3 of 20

Snake Case to Camel Case with Repository-Wide Refactor

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 213/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

APPROACH

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

HOW TO THINK ABOUT IT

Converting foo_bar to fooBar is easy. The risk is in the migration: a plain text replace also changes strings, comments, third-party names and public interfaces. Work on tokens (or the syntax tree) instead, rename only symbols defined in your own code, keep an explicit allow-list of public names, and prove nothing broke by running the tests before and after.

Basis: Written for this question; the conversion is a one-liner and the real work is a safe, token-based migration (not plain text replace).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for path in source_files(repo):
    text = read(path)
    for word in find_snake_words(text):
        if not is_third_party(word):
            new = to_camel(word)
            text = text.replace(word, new)
    write(path, text)
```

TIME	SPACE
O(size of the repository)	O(1)

WHY IT IS SLOW

Also rewrites strings, comments, third-party calls and public interface names. Renaming only the identifiers the code defines, found with the syntax tree and tokens, keeps those intact.

OPTIMAL SOLUTION · Convert names, rename only defined identifiers via tokens

TIME	SPACE
O(size of the source)	O(names)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you prove the migration is safe?

Run the full test suite before and after (green both times), compare the public API surface (exported names) before and after, and migrate in small reviewable commits with a codemod that only touches owned symbols. Anything reachable by reflection, serialisation, config files or other repositories needs an explicit alias or a deprecation period rather than a silent rename.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Snake Case to Camel Case with Repository-
# Wide Refactor
#
# Convert a valid snake_case identifier to camelCase, then
# design a safe repository-wide migration for variables
# and function names without changing third-party symbols,
# public interfaces, or the tests' ability to detect
# regressions. Implement a conversion from a valid
# snake_case identifier to camelCase. The base input is
# guaranteed valid: no leading underscore, trailing
# underscore, or doubled separator. Example: foo_bar ->
# fooBar. Extend the discussion to a repository too large
# for manual review: replace snake-case variables and
# functions across the codebase. Explain how to verify
# that the repository still works after the automated
# rewrite, including when test-package identifiers are
# rewritten too. Preserve third-party functions and public
# interfaces; explain how the migration distinguishes
# symbols that may change from contracts that must remain
# stable. Examples: foo_bar -> fooBar
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you prove the migration is safe?
#
# INTERVIEW NOTES
# Converting foo_bar to fooBar is easy. The risk is in the
# migration: a plain text replace also changes strings,
# comments, third-party names and public interfaces. Work
# on tokens (or the syntax tree) instead, rename only
# symbols defined in your own code, keep an explicit
# allow-list of public names, and prove nothing broke by
# running the tests before and after.
#
# BRUTE FORCE (say it first): Plain text replace across
# the files
# Regex-replace every snake_case word in every file. Fast
# and wrong: it rewrites strings, comments, third-party
# calls and public interface names. Time O(size of the
# repository). Space O(1).


import re
import ast
import io
import tokenize
```

```
# BRUTE FORCE (say it first): Plain text replace across
# the files | time O(size of the repository)
def snake_rewrite_brute(source):
    return re.sub(
        r"\b[a-z]+(?:_[a-z0-9]+)+\b",
        lambda m: snake_to_camel(m.group()),
        source,
    )


# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
# Idea: snake_to_camel capitalises each part after the
# first. owned_names collects the functions, arguments and
# variables the code defines (using the syntax tree),
# minus an allow-list of names that must stay.
def snake_to_camel(name):
    # foo_bar -> fooBar (input is a valid snake_case name)
    head, *rest = name.split("_")
    caps = [p[:1].upper() + p[1:] for p in rest]
    return head + "".join(caps)


# FOLLOW-UP 1: How do you prove the migration is safe?
# Answer: Run the full test suite before and after (green
# both times), compare the public API surface (exported
```

```
# names) before and after, and migrate in small reviewable
# commits with a codemod that only touches owned symbols.
def owned_names(source, keep=()):
    # names this code defines itself: safe to rename
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
            args = node.args.args + node.args.kwonlyargs
            names.update(a.arg for a in args)
        elif isinstance(node, ast.Name):
            if isinstance(node.ctx, ast.Store):
                names.add(node.id)
    # only real snake_case, never dunder or private names
    return {n for n in names if "_" in n.strip("_")
            and not n.startswith("_") and n not in keep}
```

```
# FOLLOW-UP 1: How do you prove the migration is safe?
# Answer: Run the full test suite before and after (green
# both times), compare the public API surface (exported
# names) before and after, and migrate in small reviewable
# commits with a codemod that only touches owned symbols.
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    out, prev = [], None
    # work on tokens: strings and comments are untouched
    for tok in tokenize.generate_tokens(
            io.StringIO(source).readline):
        # skip attribute names (after a dot): they may be
        # third-party, so they stay as written
        dot = prev is not None and prev.string == "."
        is_name = tok.type == tokenize.NAME
        if is_name and tok.string in owned and not dot:
            new = snake_to_camel(tok.string)
            tok = tok._replace(string=new)
        out.append(tok)
        prev = tok
    return tokenize.untokenize(out)


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    src = (
        "import os\n"
        "def load_data(file_path):\n"
        "    full_path = os.path.join(file_path, 'x_y')\n"
        "    return full_path  # load_data is fine\n"
        "result_value = load_data('a')\n")
    new = rename_identifiers(src, keep={"file_path"})
    assert "def loadData(file_path)" in new
    assert "fullPath = os.path.join" in new
    # strings, comments and third-party calls are intact
    assert "'x_y'" in new and "# load_data is fine" in new
    assert "resultValue = loadData('a')" in new
    # the migrated code still runs
    scope = {}
    exec(compile(new, "x", "exec"), scope)
    assert scope["resultValue"] == "a/x_y"
```

NOTES FROM CANDIDATES

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

PREPARATION

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | PracHub | Technical Screen | 4 of 20

Check Whether Digits Are Monotone Increasing

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 193/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy digit scan) (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/check-whether-digits-are-monotone-increasing

QUESTION

Overview: Check whether a non-negative integer's decimal digits are non-decreasing from left to right. Cover zero, single-digit values, repeated digits, and an O(d) implementation using either arithmetic digit extraction or string traversal.

APPROACH

Compare each digit with the previous one using % 10 and // 10 from the right (or string scan); handle 0 and single digits. O(d).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether equal neighbors are allowed (non-decreasing), whether the input can be negative, and whether to use the integer or string form.

BRUTE FORCE TO SAY OUT LOUD

Convert to a string and compare each pair. Fine and O(d).

OPTIMAL IDEA

Without strings: peel digits from the right with % 10 and // 10, keeping the previous digit; fail if the current digit is greater than the previous one seen to its right. O(d), O(1) space.

DRY-RUN THESE CASES

0, a single digit, 1234, 1224, 1243, 100.

LIKELY FOLLOW-UPS

Strictly increasing, or find the next monotone number.

PITFALLS

Comparing in the wrong direction when scanning right to left.

HOW TO THINK ABOUT IT

Compare neighbouring digits. You can read the digits from the right with % 10 and // 10, remembering the previous (right-hand) digit: the number is monotone if no digit is larger than the one to its right. Say first whether equal neighbours are allowed.

Basis: The check behind LeetCode 738 (non-decreasing digits).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_monotone(x):
    s = str(x)
    return s == "".join(sorted(s))
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting costs O(d log d) and copies the digits. Comparing each digit with its neighbour while peeling digits with % 10 is O(d) with O(1) space.

OPTIMAL SOLUTION · Peel digits from the right and compare with the previous one

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Strictly increasing digits.

Use >= instead of > when comparing with the previous digit, and start prev at 10 so a single digit passes.

FOLLOW-UP 2 Negative numbers or numbers in another base.

Decide what monotone means for the sign (usually apply it to the absolute value). For base b replace 10 with b in % and //.

FOLLOW-UP 3 Find the next monotone number after n.

That is the left-to-right fix from the previous question: find the first descent and copy that digit across the rest.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check Whether Digits Are Monotone
# Increasing
#
```

```
# Check whether a non-negative integer's decimal digits
# are non-decreasing from left to right. Cover zero,
# single-digit values, repeated digits, and an O(d)
# implementation using either arithmetic digit extraction
# or string traversal.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Strictly increasing digits.
# 2. Negative numbers or numbers in another base.
# 3. Find the next monotone number after n.
#
# INTERVIEW NOTES
# Compare neighbouring digits. You can read the digits
# from the right with % 10 and // 10, remembering the
# previous (right-hand) digit: the number is monotone if
# no digit is larger than the one to its right. Say first
# whether equal neighbours are allowed.
#
# BRUTE FORCE (say it first): String comparison with its
# sorted version
# Convert to a string and compare it with the sorted
# string: a number has non-decreasing digits exactly when
# its digits are already in sorted order. Clear, with an
# unneeded sort. Time O(d log d). Space O(d).


# BRUTE FORCE (say it first): String comparison with its
# sorted version | time O(d log d)
def digits_monotone_sorted(n):
    s = str(n)
    return s == "".join(sorted(s))


# BASE QUESTION (optimal): Peel digits from the right and
# compare with the previous one
# Idea: Take the last digit with n % 10, then the next
# one, and so on. Because we move right to left, the digit
# just read must not exceed the one read before it. A
# single digit and zero are monotone.
def is_monotone_digits(n):
    # non-decreasing digits, read from the right with n %
    # 10
```

```
    prev = 9
    while n:
        d = n % 10
        # a digit bigger than the one to its right: a
        # descent
        if d > prev:
            return False
        prev = d
        n //= 10
    return True


# SAME TEST on the text of the number
def is_monotone_string(n):
    # the same test on the text of the number
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))


# FOLLOW-UP 1: Strictly increasing digits.
# Answer: Use >= instead of > when comparing with the
# previous digit, and start prev at 10 so a single digit
# passes.
def is_strictly_monotone(n):
    # strictly increasing: start prev at 10, compare with
    # >=
    prev = 10
    while n:
        d = n % 10
        if d >= prev:
```

```
            return False
        prev = d
        n //= 10
    return True


# FOLLOW-UP 2: Negative numbers or numbers in another
# base.
# Answer: Decide what monotone means for the sign (usually
# apply it to the absolute value). For base b replace 10
# with b in % and //.
def is_monotone_base(n, base=10):
    # any base: use the base for % and //
    prev = base - 1
    while n:
```

```
        d = n % base
        if d > prev:
            return False
        prev = d
        n //= base
    return True


# FOLLOW-UP 3: Find the next monotone number after n.
# Answer: That is the left-to-right fix from the previous
# question: find the first descent and copy that digit
# across the rest.
def next_monotone(n):
    # smallest monotone number AFTER n: fix the first
    # descent
    d = list(str(n + 1))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            d[i + 1:] = d[i] * (len(d) - i - 1)
            break
    return int("".join(d))


if __name__ == "__main__":
    assert is_monotone_digits(0) and is_monotone_digits(7)
    assert is_monotone_digits(1223)
    assert is_monotone_digits(999)
    assert not is_monotone_digits(332)
    assert not is_monotone_digits(10)
    assert is_strictly_monotone(1234)
    assert not is_strictly_monotone(1224)
    assert is_strictly_monotone(5)
    # binary 11 -> 3 is monotone; binary 10 -> 2 is not
    assert is_monotone_base(3, 2)
    assert not is_monotone_base(2, 2)
    assert next_monotone(332) == 333
    assert next_monotone(9) == 11
    for n in range(2000):
        by_text = is_monotone_string(n)
        assert is_monotone_digits(n) == by_text
```

FUNCTION AND TEST CASES

Signature

```
def has_monotone_increasing_digits(n: int) -> bool:
SAMPLE INPUT
0
```

Test cases

Input	Expected
0	True
7	True

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | PracHub | Technical Screen | 5 of 20

Can board states be transformed?

POSTED 2026-04-01 | 202 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 93/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (inferred from overview)

Posted on PracHub: 2026-04-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/can-board-states-be-transformed

QUESTION

Overview: This question evaluates reasoning about constrained piece movement, string transformation, and invariant-based correctness in the Coding & Algorithms domain, testing skills in string manipulation and combinatorial reasoning while examining both conceptual properties and practical algorithmic implementation.

APPROACH

Invariants: the sequence of non-empty pieces must match; each L can only move left and each R only right (compare positions with two pointers).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the pieces are, which direction each may move, whether pieces may jump over each other, and what the empty cell is called.

BRUTE FORCE TO SAY OUT LOUD

BFS over all reachable board strings. Exponential but clear.

OPTIMAL IDEA

Invariants: pieces never cross, so the order of the non-empty pieces must be identical in both strings; an L can only move left, so in the start its index must be at least its index in the target; an R can only move right, so its start index must be at most its target index. Walk two pointers over the non-empty cells and check those conditions. O(n).

DRY-RUN THESE CASES

Equal strings. A different piece order. An L that would have to move right. Same pieces but different counts.

LIKELY FOLLOW-UPS

Add walls that cannot be passed or crossed, or ask for the minimum number of moves.

PITFALLS

INTERVIEW-DAY PLAN

Comparing counts only and not order. Skipping the empty cells incorrectly in the two-pointer loop.

HOW TO THINK ABOUT IT

Think invariants, not search. Pieces cannot jump each other, so the sequence of L and R letters (ignoring blanks) cannot change. Within that, an L can only go left and an R only right, so compare the position of each piece in the start and in the target. That reduces exponential BFS to one pass with two pointers.

Basis: LeetCode 777, Swap Adjacent in LR String: 'X' is an empty cell, 'L' only moves left, 'R' only moves right, pieces never pass each other.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in legal_moves(board):
            if nxt not in seen:  # XL->LX, RX->XR
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential in the length	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially. Pieces never cross, so compare the sequence of L and R pieces and check that no L moved right and no R moved left, in O(n).

OPTIMAL SOLUTION · Two pointers over the pieces

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the minimum number of moves.

Because pieces never interact beyond keeping their order, each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences when the transformation is possible.

FOLLOW-UP 2 Add walls that cannot be crossed (next question).

A wall is just another piece that never moves: it stays in the sequence of non-blank symbols and must keep the same index in both strings. The same pointer walk handles it.

FOLLOW-UP 3 Pieces can move one step per round and several move at once.

The count of rounds is the maximum over pieces of their distance, if pieces never block each other; with blocking you simulate rounds or use the per-piece arrival times with the ordering constraint.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Can board states be transformed?
#
# This question evaluates reasoning about constrained
# piece movement, string transformation, and invariant-
# based correctness in the Coding & Algorithms domain,
# testing skills in string manipulation and combinatorial
# reasoning while examining both conceptual properties and
# practical algorithmic implementation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the minimum number of moves.
# 2. Add walls that cannot be crossed (next question).
# 3. Pieces can move one step per round and several move
# at once.
#
# INTERVIEW NOTES
# Think invariants, not search. Pieces cannot jump each
# other, so the sequence of L and R letters (ignoring
# blanks) cannot change. Within that, an L can only go
# left and an R only right, so compare the position of
# each piece in the start and in the target. That reduces
# exponential BFS to one pass with two pointers.
#
# BRUTE FORCE (say it first): BFS over every reachable
# board
# From the start string, repeatedly apply the legal moves
# (XL becomes LX, RX becomes XR) and see whether the
# target appears. Correct, and the oracle for the
# invariant-based check, but the number of boards
# explodes. Time exponential in the length. Space
# exponential.


from collections import deque


# BRUTE FORCE (say it first): BFS over every reachable
# board | time exponential in the length
def can_transform_bfs(start, end):
    seen, queue = {start}, deque([start])
    while queue:
        s = queue.popleft()
        if s == end:
            return True
        for i in range(len(s) - 1):
            nxt = None
            if s[i : i + 2] == "XL":
                nxt = s[:i] + "LX" + s[i + 2 :]
            elif s[i : i + 2] == "RX":
                nxt = s[:i] + "XR" + s[i + 2 :]
            if nxt and nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False


# HELPERS for the base question
def pieces(board):
    # the non-blank symbols with their positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "X"]


# BASE QUESTION (optimal): Two pointers over the pieces
# Idea: Skip blanks in both strings and compare the next
# piece in each: they must be the same letter.
def can_transform(start, end):
    # L moves left, R moves right, X is blank. Pieces
    # never pass
    # each other, so the order of the non-X symbols is
    # fixed;
    # a '#' is a wall: it never moves and no piece crosses
    # it.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        # an L may only end up further left, an R further
        # right
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
        # a wall must stay exactly where it is
        if c1 == "#" and i != j:
            return False
    return True


# FOLLOW-UP 1: Return the minimum number of moves.
# Answer: Because pieces never interact beyond keeping
# their order, each piece moves independently from its
# start index to its target index, so the minimum is the
# sum of the absolute index differences when the
# transformation is possible.
def min_moves(start, end):
    # every piece moves on its own, one cell per move
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    return sum(abs(i - j) for (_, i), (_, j) in zip(a, b))


# FOLLOW-UP 3: Pieces can move one step per round and
# several move at once.
# Answer: The count of rounds is the maximum over pieces
# of their distance, if pieces never block each other;
# with blocking you simulate rounds or use the per-piece
# arrival times with the ordering constraint.
def rounds_needed(start, end):
    # if all pieces move at once and never block each
    # other,
    # the number of rounds is the longest single trip
    if not can_transform(start, end):
```

```
        return None
    a, b = pieces(start), pieces(end)
    trips = [abs(i - j) for (_, i), (_, j) in zip(a, b)]
    return max(trips, default=0)


if __name__ == "__main__":
    assert can_transform("RXXLRXRXL", "XRLXXRRLX")
    assert not can_transform("X", "L")
    assert can_transform("XXXL", "XXLX")
    # R can not move left, L can not move right
    assert not can_transform("XR", "RX")
    assert not can_transform("LX", "XL")
    # pieces keep their order: RL can not become LR
    assert not can_transform("RL", "LR")
    assert min_moves("RXXLRXRXL", "XRLXXRRLX") == 4
    assert min_moves("RXX", "XXR") == 2
```

```
    assert min_moves("RX", "XR") == 1
    assert min_moves("LL", "RR") is None
    # a wall splits the board: R can not jump over it
    assert can_transform("R#X", "R#X")
    assert not can_transform("R#X", "X#R")
    assert rounds_needed("RXX", "XXR") == 2
    assert rounds_needed("LL", "RR") is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(start: str, target: str) -> bool:
```

Sample call

```
solution("R_L|__L", "_RL|L__")
```

Test cases

Input	Expected
("R_L|__L", "_RL|L__")	True
("L_|R", "L|_R")	False

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

Site tags: string, simulation, medium

MEDIUM | 1Point3Acres | onsite-coding | 6 of 20

Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 202/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/document-line-wrap

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space
    would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo
    bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

APPROACH

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the width counts characters in a monospace font, how to treat explicit newlines, multiple spaces and leading spaces, what to do with a word longer than the width, and whether to return the number of lines or the lines themselves.

BRUTE FORCE TO SAY OUT LOUD

Build the text line by line by trying to add each word and measuring. That is already the greedy algorithm.

OPTIMAL IDEA

Tokenize into words and newline tokens. Keep the current line length; for each word, if it fits (with a space unless the line is empty) add it, otherwise start a new line. A newline token always ends the current line. Count lines (empty lines count). Greedy is expected; mention Knuth's minimum-raggedness DP only if asked. O(n).

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

The example giving 4. width = 1. Consecutive newlines. A word exactly the width. A word longer than the width.

LIKELY FOLLOW-UPS

Return the lines, justify the text, or minimize raggedness.

PITFALLS

Dropping empty lines. Adding a trailing space at the end of a line.

HOW TO THINK ABOUT IT

Greedy is optimal for the question 'fewest lines': put as many words on a line as fit. The decision per word is only 'fits with a space', 'fits as the first word on the line', or 'start a new line', so track the current line length, not the line text. Treat newline as a hard break and decide up front what empty lines and long words do.

Basis: Written; greedy word wrap (LeetCode 68, Text Justification, without the justification step). Assumed rules (confirm them): monospace width in characters; words are separated by whitespace; '\n' always ends a line and an empty line counts; a word longer than the width gets its own line.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wrap(text, width):
    count = 0
    for paragraph in text.split("\n"):
        line = ""
        for word in paragraph.split():
            joined = len(line) + 1 + len(word)
            if line and joined > width:
                count += 1  # finish the line
                line = word
            else:
                line = line + " " + word
                line = line.strip()
        count += 1  # last line
    return count
```

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

Re-measuring a joined string for every word costs extra copying. Keeping a running length makes the whole thing O(total characters).

OPTIMAL SOLUTION · Greedy with a running line length

TIME	SPACE
O(n)	O(1) extra beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the lines, not only the count.

Keep the same loop but append the finished line to a list (the brute force above already does it). Build each line from a list of words and join once, so each word is copied once.

FOLLOW-UP 2 A word longer than the width must be broken.

Before placing a word, while it is longer than the width flush the current line and emit width-sized chunks of the word, then continue with the remainder as a normal word.

FOLLOW-UP 3 Minimise raggedness instead of lines.

That is Knuth's word-wrap DP: dp[i] is the best cost of laying out words i.. with cost (spaces left)^2 on each line except the last; O(n * words per line). Greedy is not optimal for this objective.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Google Doc Line Wrap (Word Wrap with
# Whitespace and Newlines)
#
# Examples: text = "Hello world\nFoo bar baz qux" width =
# 10 Line 1: "Hello" (5 chars; "world" + space would be 11
# -> break) Line 2: "world" Line 3: "Foo bar" (7 chars;
# "baz" -> "Foo bar baz" = 11 -> break) Line 4: "baz qux"
# Output: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the lines, not only the count.
# 2. A word longer than the width must be broken.
# 3. Minimise raggedness instead of lines.
#
# INTERVIEW NOTES
# Greedy is optimal for the question 'fewest lines': put
# as many words on a line as fit. The decision per word is
# only 'fits with a space', 'fits as the first word on the
# line', or 'start a new line', so track the current line
# length, not the line text. Treat newline as a hard break
# and decide up front what empty lines and long words do.
#
# BRUTE FORCE (say it first): Build each line as a string
# and measure it
# Try appending each word to the current line string and
# check its length, starting a new line when it exceeds
# the width. Correct, but joining strings repeatedly costs
# more than the length counter in the optimal version.
# Time O(total characters * words per line). Space
# O(width).


# BRUTE FORCE (say it first): Build each line as a string
# and measure it | time O(total characters * words per
# line)
def wrap_lines_brute(text, width):
    lines = []
    for paragraph in text.split("\n"):
        words = paragraph.split()
        cur = ""
        for w in words:
            trial = (cur + " " + w) if cur else w
            if len(trial) <= width or not cur:
                cur = trial
            else:
                lines.append(cur)
                cur = w
        lines.append(cur)
    return lines


# HELPERS for the base question
# FOLLOW-UP 1: Return the lines, not only the count.
# Answer: Keep the same loop but append the finished line
# to a list (the brute force above already does it). Build
# each line from a list of words and join once, so each
# word is copied once.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def wrap_lines(text, width, break_long=False):
    lines = []
    # a newline always starts a new paragraph (and a new
    # line)
    for paragraph in text.split("\n"):
        cur = ""
        words = paragraph.split()
        if break_long:
            pieces = [split_long(w, width) for w in words]
            words = [p for chunk in pieces for p in chunk]
        for w in words:
            if not cur:
                cur = w
            elif len(cur) + 1 + len(w) <= width:
                # the word and one space still fit
                cur += " " + w
            else:
                lines.append(cur)
                cur = w
        # the last (or only, or empty) line of the
        # paragraph
        lines.append(cur)
    return lines


# FOLLOW-UP 2: A word longer than the width must be
# broken.
# Answer: Before placing a word, while it is longer than
# the width flush the current line and emit width-sized
# chunks of the word, then continue with the remainder as
# a normal word.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_long(word, width):
    # a word longer than the line is cut into pieces
    starts = range(0, len(word), width)
    return [word[i:i + width] for i in starts]


# BASE QUESTION (optimal): Greedy with a running line
# length
# Idea: Split on newlines first (each paragraph ends its
# line), then for each word compare cur_len + 1 +
# len(word) with the width. Only the length is tracked,
# never the joined line.
def wrap_count(text, width):
    return len(wrap_lines(text, width))


# FOLLOW-UP 3: Minimise raggedness instead of lines.
# Answer: That is Knuth's word-wrap DP: dp[i] is the best
# cost of laying out words i.. with cost (spaces left)^2
# on each line except the last; O(n * words per line).
# Greedy is not optimal for this objective.
```

```python
def min_raggedness(words, width):
    # Knuth: minimise the squared spare space of every
    # line
    # except the last; greedy is NOT optimal for this goal
    n = len(words)
    INF = float("inf")
    # cost[i]: best total cost for words[i:]
    cost, nxt = [INF] * (n + 1), [0] * (n + 1)
    cost[n] = 0
    for i in range(n - 1, -1, -1):
        length = -1
        for j in range(i, n):
            length += len(words[j]) + 1
            if length > width:
                break
            spare = width - length
            # the last line costs nothing
            extra = 0 if j == n - 1 else spare ** 2
            if extra + cost[j + 1] < cost[i]:
                cost[i] = extra + cost[j + 1]
                nxt[i] = j + 1
    lines, i = [], 0
```

```python
    while i < n:
        lines.append(" ".join(words[i:nxt[i]]))
        i = nxt[i]
    return cost[0], lines


if __name__ == "__main__":
    text = "Hello world\nFoo bar baz qux"
    assert wrap_lines(text, 10) == [
        "Hello", "world", "Foo bar", "baz qux"]
    assert wrap_count(text, 10) == 4
    # blank lines count as lines
    assert wrap_count("a\n\nb", 5) == 3
    # a long word is split when asked
    cut = wrap_lines("abcdefgh", 3, True)
    assert cut == ["abc", "def", "gh"]
    assert wrap_lines("abcdefgh", 3) == ["abcdefgh"]
    # the best layout can beat the greedy one
    cost, lines = min_raggedness(
        "aaa bb cc ddddd".split(), 6)
    assert lines == ["aaa", "bb cc", "ddddd"]
    assert cost == 9 + 1
```

NOTES FROM CANDIDATES

Single-pass tokenizer + accumulator: track curLen; on each token, decide "fits with leading space", "fits without space" (start of line), or "flush and start new line".
Watch out for \n right after another \n (empty line counts), \n at the end of input, leading whitespace, multiple consecutive spaces inside a line.
Don't try to handle font / kerning / hyphenation — the prompt assumes monospace width counted in characters.
Greedy packing is the expected answer (LC 68 style). Mention DP word-wrap (Knuth's algorithm) only if the interviewer pushes on 'minimum raggedness'; greedy is optimal-enough and what graders look for. Watch edge cases: a single word longer than W, an explicit newline forcing a break mid-pack, trailing whitespace handling.

PREPARATION

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | PracHub | Technical Screen | 7 of 20

Decide string transform with directional tokens and walls

POSTED 2025-07-15 | 462 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 94/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (with X walls)

Posted on PracHub: 2025-07-15 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/decide-string-transform-with-directional-tokens-and-walls

QUESTION

You are given two equal-length strings start and target of length n, each consisting only of the characters 'L', 'R', '.', and 'X'. A dot '.' represents an empty slot; 'X' represents an immovable wall; 'L' and 'R' are tokens that may move across empty slots subject to these rules: (1) An 'L' token may move any number of positions left into '.', but may never move right, pass through another token, or pass through an 'X'. (2) An 'R' token may move any number of positions right into '.', but may never move left, pass through another token, or pass through an 'X'. (3) Only one token can occupy a position at any time. Determine whether target can be obtained from start by a sequence of valid moves. If yes, return true; otherwise, return false. Design an algorithm that runs in O(n) time and O(1) extra space beyond a few pointers/counters. Explain your invariants, prove correctness informally, analyze time and space complexity, and walk through a few tricky edge cases (e.g., mismatched wall layouts, blocks separated by walls, consecutive tokens with no gaps).

APPROACH

Split both strings at 'X' walls (must align), then within each segment apply the L/R two-pointer rule: L only moves left, R only right, order of pieces preserved.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what each symbol means (L moves left, R moves right, dot empty, X wall that never moves and cannot be crossed), whether a token can jump over another (no), and whether both strings have the same length.

BRUTE FORCE TO SAY OUT LOUD

BFS over all board states by moving tokens one step. Exponential.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Compare the sequences of non-dot characters: they must be identical, which also forces every X to appear in the same order. Then with two pointers over those tokens check that each L has start index at least target index, each R has start index at most target index, and each X has the same index in both. Walls cannot be crossed because order is preserved. O(n).

DRY-RUN THESE CASES

Identical strings, an L that would move right, an R blocked by a wall, a different number of tokens, a wall at a different index.

LIKELY FOLLOW-UPS

Return the minimum number of moves (sum of index differences), or the move sequence.

PITFALLS

Letting a token cross a wall. Checking only counts of L and R.

HOW TO THINK ABOUT IT

Walls are pieces that never move. Ignore the dots and compare the sequences of L, R and X: they must match in order and in count, and each X must be at the same index in both strings. For the L and R pieces keep the same rule: L never moves right, R never moves left. Order is preserved, so no piece can get across a wall.

Basis: LeetCode 777 with walls: 'L' moves left, 'R' moves right, '.' is empty, 'X' is an immovable wall; no piece crosses another piece or a wall.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in one_step(board):
            if nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially with the length. Comparing the non-dot sequence (walls and pieces in order) with position rules (L never moves right, R never moves left, walls stay) is O(n).

OPTIMAL SOLUTION · Compare the non-dot sequences with position rules

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Segments between walls are independent.

You can also split both strings at the walls (they must be at identical positions) and apply the L/R rule to each segment separately. The single pass above is equivalent.

FOLLOW-UP 2 Return the moves or the minimum number of steps.

Per piece, the steps are the index difference; list moves by letting each piece walk one cell at a time in an order that never blocks (move L pieces left to right and R pieces right to left).

FOLLOW-UP 3 The board is 2D.

Then pieces can pass each other by going around, and invariants like order no longer hold. Ask for the exact movement rules; most likely you would model it as a graph search with a heuristic.

FOLLOW-UP 4 Return the minimum number of moves.

When the transformation is possible each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences (checked against BFS on small strings).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Decide string transform with directional
# tokens and walls
#
# You are given two equal-length strings start and target
# of length n, each consisting only of the characters 'L',
# 'R', '.', and 'X'. A dot '.' represents an empty slot;
# 'X' represents an immovable wall; 'L' and 'R' are tokens
# that may move across empty slots subject to these rules:
# ( 1) An 'L' token may move any number of positions left
# into '.', but may never move right, pass through another
# token, or pass through an 'X'. ( 2) An 'R' token may
# move any number of positions right into '.', but may
# never move left, pass through another token, or pass
# through an 'X'. ( 3) Only one token can occupy a
# position at any time. Determine whether target can be
# obtained from start by a sequence of valid moves. If
# yes, return true; otherwise, return false. Design an
# algorithm that runs in O(n) time and O( 1) extra space
# beyond a few pointers/counters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Segments between walls are independent.
# 2. Return the moves or the minimum number of steps.
# 3. The board is 2D.
# 4. Return the minimum number of moves.
#
# INTERVIEW NOTES
# Walls are pieces that never move. Ignore the dots and
# compare the sequences of L, R and X: they must match in
# order and in count, and each X must be at the same index
# in both strings. For the L and R pieces keep the same
# rule: L never moves right, R never moves left. Order is
# preserved, so no piece can get across a wall.
#
# BRUTE FORCE (say it first): BFS over boards with walls
# Apply the legal single-step moves ('.L' becomes 'L.',
# 'R.' becomes '.R'), walls never move, and search for the
# target. Exact, so it is the oracle for the pointer
# check, but it explodes with the length. Time
# exponential. Space exponential.


from collections import deque
```

```
# BRUTE FORCE over real moves (checker)
def bfs_moves(start, end):
    # brute force over real moves (the check used by the
    # demo)
    seen, queue = {start: 0}, deque([start])
    while queue:
        s = queue.popleft()
        if s == end:
            return seen[s]
        for i, ch in enumerate(s):
            for step, piece in ((-1, "L"), (1, "R")):
                j = i + step
                inside = 0 <= j < len(s)
                if ch == piece and inside and s[j] == ".":
                    t = list(s)
                    t[i], t[j] = ".", ch
                    t = "".join(t)
                    if t not in seen:
                        seen[t] = seen[s] + 1
                        queue.append(t)
    return None


# HELPERS for the base question
def pieces(board):
    # everything except the empty '.' slots, with
    # positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "."]


# BASE QUESTION (optimal): Compare the non-dot sequences
# with position rules
# Idea: Walk both strings skipping dots. The next symbol
# must be equal; a wall must be at the same index; an L
# must satisfy start index >= target index; an R must
# satisfy start index <= target index.
def can_transform(start, end):
    # 'L' moves left, 'R' moves right, 'X' is an immovable
    # wall. Pieces never pass each other, so their order
    # is
    # fixed; each one only has a direction it may move.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
```

```
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        if c1 == "X" and i != j:
            return False
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
    return True


# MIN MOVES (pieces move independently)
# FOLLOW-UP 4: Return the minimum number of moves.
# Answer: When the transformation is possible each piece
# moves independently from its start index to its target
# index, so the minimum is the sum of the absolute index
# differences (checked against BFS on small strings).
def min_moves(start, end):
    # pieces move independently: the sum of index
    # differences
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    return sum(abs(i - j) for (_, i), (_, j) in zip(a, b))


if __name__ == "__main__":
    assert can_transform("R.L", ".RL") is True
    assert not can_transform("RL.", "LR.")
    # a wall cannot move and splits the board
    assert can_transform("R.X.L", ".RX.L") is True
    assert not can_transform("R.X", "X.R")
    assert min_moves("R..L", ".RL.") == 2
    assert min_moves("L.", ".L") is None
    import itertools
    boards = []
    for n in range(1, 5):
        for p in itertools.product("LR.X", repeat=n):
            boards.append("".join(p))
    for s in boards:
        for t in boards:
            if len(s) == len(t) and len(s) <= 4:
```

```
                fast = min_moves(s, t)
                assert fast == bfs_moves(s, t), (s, t)
```

FUNCTION AND TEST CASES

Signature

```
def canTransform(start: str, target: str) -> bool:
```

Sample call

```
canTransform('R...L', 'RL...')
```

Test cases

Input	Expected
`('R...L', 'RL...')`	`True`
`('R.L', 'R..L')`	`False`

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | IN APP LC 249 | Exponent | 8 of 20

Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

ASKED (ANY CO.) ~8 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 35.4 | 30 DAYS 0 | PRIORITY 109/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: your app's interview approach (LeetCode #249)

LeetCode match: LC 249 Group Shifted Strings (inferred from short statement)

Last asked (any company): 8 months ago, about 2026-02 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6356/group-buddy-strings-same-distance

QUESTION

Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

APPROACH

Normalize each string by its successive character differences (mod 26) as a tuple key, then group by key in a hash map. O(total chars).

HOW TO THINK ABOUT IT

Buddies are shifted copies of each other, so what they share is the shape: the gaps between neighbouring letters. Turn each string into its shape (a tuple of gaps mod 26) and group by it with a dictionary; you never compare two strings directly. Mention that single letters all have the same (empty) shape.

Basis: LeetCode 249, Group Shifted Strings.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(words):
    groups = []
    for s in words:
        for group in groups:
            if is_shift(group[0], s):
                group.append(s)
                break
        else:
            groups.append([s])
    return groups
def is_shift(a, b):
    if len(a) != len(b):
        return False
    shifts = {(ord(y) - ord(x)) % 26
              for x, y in zip(a, b)}
    return len(shifts) <= 1
```

TIME	SPACE
O(n * groups * L)	O(n)

WHY IT IS SLOW

Each string is compared with the representative of every group, O(n * groups * L). Using the sequence of gaps between neighbouring letters as a dictionary key groups everything in one pass.

OPTIMAL SOLUTION · Group by the tuple of successive differences

TIME	SPACE
O(total characters)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if shifting is not cyclic (z does not wrap to a)?

Drop the % 26 and use the raw differences as the key. Two strings with the same raw gaps differ by one fixed integer shift, and both are already inside 'a'..'z', so that shift is automatically legal: the gap tuple alone decides buddies, with no extra range check. What changes is that 'az' and 'ba' are no longer buddies (gaps 25 and -1 instead of 25 and 25).

FOLLOW-UP 2 How would you do this with strings that are too many for one machine?

Compute the gap key on each machine, hash-partition the strings by key (so every buddy lands on the same worker), and group locally. The key is the only thing exchanged, so shuffle cost is one pass over the data. **Cost:** O(total characters / machines) per machine to compute the keys, plus one shuffle of the keys; each group is then built locally in O(group size).

FOLLOW-UP 3 Shifting does not wrap around (z stays z).

Use the raw gaps (no % 26) as the key. Equal raw gaps mean one fixed integer shift, which is automatically legal because both strings are already letters, so no extra check is needed.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Given an array of strings, return groups
# of "buddy strings," where the strings have the same
# length and the distance between corresponding characters
# is the same.
#
# Given an array of strings, return groups of "buddy
# strings," where the strings have the same length and the
# distance between corresponding characters is the same.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if shifting is not cyclic (z does not wrap to
# a)?
# 2. How would you do this with strings that are too many
# for one machine?
# 3. Shifting does not wrap around (z stays z).
#
# INTERVIEW NOTES
# Buddies are shifted copies of each other, so what they
# share is the shape: the gaps between neighbouring
# letters. Turn each string into its shape (a tuple of
# gaps mod 26) and group by it with a dictionary; you
# never compare two strings directly. Mention that single
# letters all have the same (empty) shape.
#
# BRUTE FORCE (say it first): Compare each string with one
# representative of every existing group
# For each string look through the existing groups and
# test it against the group's first member (same length
# and a constant shift); start a new group if none fits.
# Correct and the oracle; each string is compared with
# many groups. Time O(n * groups * L). Space O(n).


from collections import defaultdict


# BRUTE FORCE (say it first, it also checks the answers)
def are_buddies(a, b):
    # oracle: is there one shift that turns a into b?
    if len(a) != len(b):
        return False
    pairs = zip(a, b)
    shifts = {(ord(y) - ord(x)) % 26 for x, y in pairs}
    return len(shifts) <= 1
```

```
# HELPERS for the base question
def shift_key(word):
    # the gaps between neighbouring letters, wrapping z to
    # a.
    # Words that are shifts of each other share this key.
    return tuple((ord(b) - ord(a)) % 26
                 for a, b in zip(word, word[1:]))


# BASE QUESTION (optimal): Group by the tuple of
# successive differences
# Idea: Two strings are buddies when every letter is
# shifted by the same amount, which means the gaps between
# neighbouring letters (mod 26) are identical. Use those
# gaps as the hash key.
def group_buddies(words, key=shift_key):
    # words of different length have keys of different
    # length, so they never land in the same group
    groups = defaultdict(list)
    for w in words:
        groups[(len(w), key(w))].append(w)
    return list(groups.values())


# FOLLOW-UP 1: What if shifting is not cyclic (z does not
# wrap to a)?
# Answer: Drop the % 26 and use the raw differences as the
# key.
# FOLLOW-UP 3: Shifting does not wrap around (z stays z).
# Answer: Use the raw gaps (no % 26) as the key. Equal raw
# gaps mean one fixed integer shift, which is
# automatically legal because both strings are already
# letters, so no extra check is needed.
def raw_key(word):
    # follow-up: no wrap around (z stays z). The raw gaps
    # already decide it: one fixed integer shift is legal
    # because both words are already made of letters.
    return tuple(ord(b) - ord(a)
                 for a, b in zip(word, word[1:]))


# FOLLOW-UP 2: How would you do this with strings that are
# too many for one machine?
# Answer: Compute the gap key on each machine, hash-
# partition the strings by key (so every buddy lands on
# the same worker), and group locally. The key is the only
# thing exchanged, so shuffle cost is one pass over the
# data.
def group_distributed(words, machines, key=shift_key):
    # follow-up: too many words for one machine. Each
    # machine
    # computes keys for its slice, every pair (key, word)
    # is
    # sent to machine hash(key) % machines, and each
    # groups what it receives. Only the key travels.
    slices = [words[i::machines] for i in range(machines)]
    inbox = [[] for _ in range(machines)]
    for part in slices:
        for w in part:
            k = (len(w), key(w))
            inbox[hash(k) % machines].append((k, w))
    result = []
    for mail in inbox:
        local = defaultdict(list)
        for k, w in mail:
            local[k].append(w)
        result.extend(local.values())
    return result


# HELPERS for the checks below
def canonical(groups):
    return sorted(sorted(g) for g in groups)


if __name__ == "__main__":
    words = ["abc", "bcd", "acef", "xyz", "az", "ba",
             "a", "z"]
    got = canonical(group_buddies(words))
    assert got == [["a", "z"], ["abc", "bcd", "xyz"],
                   ["acef"], ["az", "ba"]]
    # without wrap around "az" and "ba" are different
    plain = canonical(group_buddies(words, raw_key))
    assert ["az", "ba"] not in plain
```

```
    assert ["abc", "bcd", "xyz"] in plain
    assert group_buddies([]) == []
    import random
    for _ in range(200):
        ws = []
```

```
        for _ in range(random.randint(0, 10)):
            size = random.randint(1, 3)
            ws.append("".join(random.choice("abyz")
                              for _ in range(size)))
        groups = group_buddies(ws)
        assert sum(len(g) for g in groups) == len(ws)
        for g in groups:
            assert all(are_buddies(g[0], w) for w in g)
        # two different groups are never buddies
        for g in groups:
            for h in groups:
                if g is not h:
                    assert not are_buddies(g[0], h[0])
        parts = random.randint(1, 4)
        mine = group_distributed(ws, parts)
        assert canonical(mine) == canonical(groups)
```

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | IN APP LC 271 | Exponent | 9 of 20

Write functions to serialize and deserialize a list of strings.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 28.2 | 30 DAYS 0 | PRIORITY 129/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: your app's interview approach (LeetCode #271)

LeetCode match: LC 271 Encode and Decode Strings

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | 2 interviews

Original:
https://www.aced.io/questions/1307/serialize-deserialize-list-strings

QUESTION

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples lst = ['hello', 'world'] serialized = serialize(lst)

serialized output

: "\x05hello\x05world" deserialized = deserialize(serialized)

deserialized output should be

: ['hello', 'world'] lst = [''] serialized = serialize(lst)

serialized output

: "\x00" deserialized = deserialize(serialized)

deserialized output should be

APPROACH

Length-prefix encoding (e.g. '5#hello'), decode by reading the length and slicing; safe for any characters including delimiters.

HOW TO THINK ABOUT IT

Any text can appear inside a string, so a delimiter is never safe by itself. The fix is to say how long each string is before you write it (length prefix), so the decoder never has to look for a separator inside the data. Always check two corner cases: the empty list and a list containing the empty string.

Basis: LeetCode 271, Encode and Decode Strings.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def serialize(items):
    return ",".join(items)
def deserialize(text):
    return text.split(",")  # breaks on commas
```

TIME	SPACE
O(total length)	O(total length)

WHY IT IS SLOW

It is fast (O(total length)) but wrong: a string that contains the delimiter breaks it, and an empty list cannot be told from a list of one empty string. Writing each string's length before it fixes both.

OPTIMAL SOLUTION · Length-prefix every string

TIME	SPACE
O(total length)	O(total length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use escaping instead of a length prefix.

Escape the separator and the escape character inside each string, then terminate each item with the unescaped separator. The decoder tracks whether the previous character was an escape. It is streamable, while the length prefix can be read in O(1) jumps.

FOLLOW-UP 2 The strings are long or binary.

Use bytes and a fixed-width length (for example a 4-byte big-endian integer from struct.pack). Fixed-width lengths avoid the digit-scanning loop and work for any byte value.

FOLLOW-UP 3 Just use JSON?

Yes if the format is free to choose: json.dumps handles quoting and Unicode correctly. Mention it as the practical answer, but code the length-prefix version when the interviewer wants to see you handle the delimiter problem yourself.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Write functions to serialize and
# deserialize a list of strings.
#
# Serialize and Deserialize a List of Strings. You are
# tasked with writing two functions: serialize and
# deserialize. The serialize function will take a list of
# strings as input and return a single string. The
# deserialize function will take this single string and
# reconstruct the original list of strings. Details The
# serialize function should concatenate all strings in the
# list into a single string, with each original string's
# length encoded as a single character prefix. The
# deserialize function should be able to take the single
# string and accurately split it back into the original
# list of strings, using the encoded length characters as
# guides. For example, consider a list ["hello", "world"].
# The serialization of this list might be
# "\x05hello\x05world", where \x05 indicates that the
# following string has a length of 5 characters.
# Constraints The maximum length of any string is 65535
# (0xFFFF).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use escaping instead of a length prefix.
# 2. The strings are long or binary.
# 3. Just use JSON?
#
# INTERVIEW NOTES
# Any text can appear inside a string, so a delimiter is
# never safe by itself. The fix is to say how long each
# string is before you write it (length prefix), so the
# decoder never has to look for a separator inside the
# data. Always check two corner cases: the empty list and
# a list containing the empty string.
#
# BRUTE FORCE (say it first): Join with a delimiter
# Join with a separator like ',' and split on decode. It
# works only while no string contains the separator, and
# it cannot tell an empty list from a list with one empty
# string. Time O(total length). Space O(total length).


import json
import struct
```

```
# BRUTE FORCE (say it first): Join with a delimiter | time
# O(total length)
def encode_join(strs):
    return ",".join(strs)


# BASE QUESTION (optimal): Length-prefix every string
# Idea: Write len(s), a separator, then s itself. To
# decode read digits up to the separator to get the
# length, then take exactly that many characters.
```

```
def serialize(items):
    # length prefix: "<length>#<text>" for every string.
    # The
    # reader knows exactly how much to read, so the text
    # may
    # hold any character, even "#" and digits.
    return "".join(f"{len(s)}#{s}" for s in items)


# BASE QUESTION (optimal): Length-prefix every string
# Idea: Write len(s), a separator, then s itself. To
# decode read digits up to the separator to get the
# length, then take exactly that many characters.
def deserialize(text):
    items, i = [], 0
    while i < len(text):
        # read the digits up to the "#"
        sep = text.index("#", i)
        size = int(text[i:sep])
        start = sep + 1
        items.append(text[start:start + size])
        i = start + size
    return items


# FOLLOW-UP 1: Use escaping instead of a length prefix.
# Answer: Escape the separator and the escape character
# inside each string, then terminate each item with the
# unescaped separator. The decoder tracks whether the
# previous character was an escape.
def serialize_escaped(items):
    # follow-up: escaping. Inside an item "\" becomes "\\"
    # and "," becomes "\,"; every item ends with a plain
    # ",".
```

```
    out = []
    for s in items:
        s = s.replace("\\", "\\\\").replace(",", "\\,")
        out.append(s + ",")
    return "".join(out)


# FOLLOW-UP 1: Use escaping instead of a length prefix.
# Answer: Escape the separator and the escape character
# inside each string, then terminate each item with the
# unescaped separator. The decoder tracks whether the
# previous character was an escape.
def deserialize_escaped(text):
    items, cur, escaped = [], [], False
    for ch in text:
        if escaped:
            # the character after "\" is taken literally
            cur.append(ch)
            escaped = False
        elif ch == "\\":
            escaped = True
        elif ch == ",":
            items.append("".join(cur))
            cur = []
        else:
            cur.append(ch)
    if cur or escaped:
        raise ValueError("truncated input")
    return items


# FOLLOW-UP 2: The strings are long or binary.
# Answer: Use bytes and a fixed-width length (for example
# a 4-byte big-endian integer from struct.pack). Fixed-
# width lengths avoid the digit-scanning loop and work for
# any byte value.
def serialize_bytes(items):
    # follow-up: long or binary data. A fixed 4 byte
```

```
    # length
    # (big-endian) before each item: no digits to scan.
    out = bytearray()
    for s in items:
        raw = s
        if isinstance(s, str):
            raw = s.encode("utf-8")
```

```
        out += struct.pack(">I", len(raw)) + raw
    return bytes(out)


# FOLLOW-UP 2: The strings are long or binary.
# Answer: Use bytes and a fixed-width length (for example
# a 4-byte big-endian integer from struct.pack). Fixed-
# width lengths avoid the digit-scanning loop and work for
# any byte value.
def deserialize_bytes(data):
    items, i = [], 0
    while i < len(data):
        (size,) = struct.unpack(">I", data[i:i + 4])
        items.append(data[i + 4:i + 4 + size])
        i += 4 + size
    return items


# FOLLOW-UP 3: Just use JSON?
# Answer: Yes if the format is free to choose: json.dumps
# handles quoting and Unicode correctly.
def serialize_json(items):
    # follow-up: the practical answer when the format is
    # free
    return json.dumps(items)


# FOLLOW-UP 3: Just use JSON?
# Answer: Yes if the format is free to choose: json.dumps
# handles quoting and Unicode correctly.
def deserialize_json(text):
    return json.loads(text)


if __name__ == "__main__":
    tricky = ["", "a", "12#x", "hello, world", "\\", ",,",
              "é??", "3#abc", "\n"]
    for data in ([], [""], ["a"], tricky):
        assert deserialize(serialize(data)) == data
        assert deserialize_escaped(
            serialize_escaped(data)) == data
        back = deserialize_json(serialize_json(data))
        assert back == data
        raw = deserialize_bytes(serialize_bytes(data))
        assert [r.decode("utf-8") for r in raw] == data
```

```
    assert serialize(["ab", "c"]) == "2#ab1#c"
    assert serialize_escaped(["a,b"]) == "a\\,b,"
    assert serialize_escaped([""]) == ","
    assert serialize_escaped([]) == ""
    assert serialize_bytes(["a"]) == b"\x00\x00\x00\x01a"
    # binary data that is not text works too
    blob = [bytes([0, 255, 10])]
    packed = serialize_bytes(blob)
    assert deserialize_bytes(packed) == blob
    try:
        deserialize_escaped("abc")
        raise AssertionError("should fail")
    except ValueError:
        pass
```

OFFICIAL PAGE TEXT

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples

```
lst = ['hello', 'world']
serialized = serialize(lst)
# serialized output: "\x05hello\x05world"
deserialized = deserialize(serialized)
# deserialized output should be: ['hello', 'world']

lst = ['']
serialized = serialize(lst)
# serialized output: "\x00"
deserialized = deserialize(serialized)
# deserialized output should be: ['']

lst = ['a', 'bc', 'def']
serialized = serialize(lst)
# serialized output: "\x01a\x02bc\x03def"
deserialized = deserialize(serialized)
# deserialized output should be: ['a', 'bc', 'def']
```

OFFICIAL PAGE TEXT

Our solution serializes and deserializes a list of strings by encoding the length of each string as a single character and concatenating it with the original string. The serialize function processes the input list of strings by iterating through each string. For every string, it first encodes the length of the string as a single character using the encodeNumber function. The encoded length is then appended to a result list, followed by the string itself. The result list is ultimately joined into a single string, which represents the serialized form of the list of strings. The deserialize function takes the serialized string and reconstructs the original list of strings. It starts by iterating through the serialized string, decoding the length of each string using the decodeNumber function, then extracting the string of that length from the serialized data. These strings are collected in a list and returned as the final deserialized result. Time Complexity: The time complexity of both serialize and deserialize is O(n), where n is the total number of characters across all strings in the input list. In serialize, we iterate through every string, performing constant-time operations (encoding the length and appending the string) for each string. Similarly, in deserialize, we iterate through the serialized string, decoding lengths and extracting substrings, which are also constant-time operations for each step. Space Complexity:

The space complexity is O(n) for both functions since we store the serialized form of the input list (which has a size proportional to the total number of characters across all strings) and reconstruct the list in deserialize.Daniel answers "Write a pair of functions to serialize and deserialize a list of strings."No answers contributed by the community yet.Related questionsWrite a function to do Excel-style operations.Write a function to check whether a string contains a substring.Given a list of bounding boxes and annotations, write a function to compute Intersection over Union.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to

run code

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | Exponent | 10 of 20

Determine if two sentences are similar.

ASKED (ANY CO.) ~4 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 21 | 30 DAYS 0 | PRIORITY 144/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1813 Sentence Similarity III

Last asked (any company): 4 years ago, about 2022-10 (Exponent shows only a rough age) | 1 interview

Original: https://www.aced.io/questions/2280/determine-similar-sentences

QUESTION

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples sentence1 = ["Let's", "code", "in", "Python"] sentence2 = ["Let's", "program", "in", "Python"] similarPairs = [("code", "program"),] output: true sentence1 = ["I", "love", "to", "play", "football"] sentence2 = ["I", "love", "playing", "soccer"] similarPairs = [("play", "playing"), ("football", "soccer")] output: false, different sentence lengths sentence1 = ["Do", "you", "like", "coffee"] sentence2 = ["Do", "you", "love", "coffee"] similarPairs = [("like", "enjoy"), ("coffee", "tea"),] output: false

"like" is not similar to

"love" based on the given pairs. sentence1 = ["I", "really", "love", "leetcode", "and", "apples"] sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"] similarPairs = [("very", "so"), ("love", "adore"), ("really", "very"), ("leetcode", "codesignal"), ("apples", "oranges"), ("like", "adore"),] output: trueThe provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are c

APPROACH

Two pointers: match the longest common prefix and suffix of the word lists; similar iff the shorter sentence is fully covered by prefix + suffix.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which version: similarity from a list of word pairs (LeetCode 734 or 737), and whether similarity is transitive (737) or not (734). If instead one sentence is the other with a block inserted in the middle (LeetCode 1813), say so and solve that. Ask about case and equal length.

BRUTE FORCE TO SAY OUT LOUD

For each position check the pair against the list by scanning it. O(n * P).

OPTIMAL IDEA

Put the pairs in a set (both directions) for the non-transitive version: lengths must match and each word pair is equal or in the set, O(n + P). For the transitive version build a union-find over words and compare roots. For the insert-a-block version use two pointers to match the common prefix and the common suffix; the sentences are similar if the shorter one is fully covered.

DRY-RUN THESE CASES

Different lengths, identical sentences, a pair that is only transitively similar (decides the version), duplicate pairs.

LIKELY FOLLOW-UPS

Make similarity transitive, add weights, or allow at most one unmatched word.

PITFALLS

Assuming similarity is symmetric when the pairs are one-directional. Using lists instead of sets.

HOW TO THINK ABOUT IT

Ask which similarity you have: from a pair list, transitive or not. Without transitivity it is a set lookup per word pair. With transitivity, similar words form groups, which is union-find (or BFS on a word graph). The variant where a block is inserted is a different problem: match the common prefix and suffix of the word lists.

Basis: LeetCode 734 (similarity from a list of word pairs), 737 (the same with transitive similarity) and 1813 (one sentence is the other with a block inserted); the question text matches 734/737.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    for a, b in zip(s1, s2):
        if a == b:
            continue
        if [a, b] in pairs or [b, a] in pairs:
            continue
        return False
    return True
```

BRUTE FORCE

TIME	SPACE
O(n * P)	O(1)

WHY IT IS SLOW

Each word pair scans the whole pair list, O(n * P). Putting the pairs into a set makes each check O(1).

OPTIMAL SOLUTION · Set of pairs (not transitive)

TIME	SPACE
O(n + P)	O(P)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Similarity is transitive (a ~ b and b ~ c means a ~ c).

Union-find over the words: union each pair, then two words are similar if they are equal or have the same root. Path compression keeps it near O(n + P).

FOLLOW-UP 2 One sentence is the other with a sentence of words inserted somewhere (LeetCode 1813).

Match the longest common prefix and the longest common suffix of the word lists; the shorter list must be covered by prefix plus suffix.

FOLLOW-UP 3 Case should not matter, or words can have synonyms with scores.

Normalise with casefold before hashing. For scores, a graph with weighted edges and a threshold query turns it into reachability along strong edges.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine if two sentences are similar.
#
# Determine if Two Sentences Are Similar. Two sentences
# are similar if they have the same length and each pair
# of corresponding words in the two sentences is similar.
# The similarity between words is defined by the provided
# list of similar word pairs. A word is always similar to
# itself. For example, if we have the list of similar word
# pairs as [("great", "good"), ("acting","drama"),
# ("skills","talent")], then the sentences "You have great
# acting skills" and "You have good drama talent" are
# similar.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Similarity is transitive (a ~ b and b ~ c means a ~
# c).
# 2. One sentence is the other with a sentence of words
# inserted somewhere (LeetCode 1813).
# 3. Case should not matter, or words can have synonyms
# with scores.
#
# INTERVIEW NOTES
# Ask which similarity you have: from a pair list,
# transitive or not. Without transitivity it is a set
# lookup per word pair. With transitivity, similar words
# form groups, which is union-find (or BFS on a word
# graph). The variant where a block is inserted is a
# different problem: match the common prefix and suffix of
# the word lists.
#
# BRUTE FORCE (say it first): Scan the pair list for each
# word pair
# For each position, check equality or scan the whole list
# of similar pairs in both orders. Correct, but every word
# pair walks the entire pair list. Time O(n * P). Space
# O(1).


from collections import defaultdict, deque


# BRUTE FORCE (say it first, it also checks the answers)
def reachable(a, b, pairs):
    # oracle: breadth first search over the similarity
    # graph
    graph = defaultdict(set)
    for x, y in pairs:
        graph[x].add(y)
        graph[y].add(x)
    seen, queue = {a}, deque([a])
    while queue:
        cur = queue.popleft()
        if cur == b:
            return True
        for nxt in graph[cur] - seen:
            seen.add(nxt)
            queue.append(nxt)
    return False


# HELPERS for follow-up 1
class DisjointSet:
    def __init__(self):
        self.parent = {}

    def find(self, x):
        self.parent.setdefault(x, x)
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        # path compression: point everything on the way up
        # straight at the root
        while self.parent[x] != root:
            self.parent[x], x = root, self.parent[x]
        return root

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# BASE QUESTION (optimal): Set of pairs (not transitive)
# Idea: Put each pair into a set in both directions, so a
# word pair is similar if the words are equal or the pair
# is in the set. Sentences of different length are never
# similar.
def similar_direct(s1, s2, pairs):
    # same length, and every pair of words is equal or a
    # listed pair (in either direction)
    if len(s1) != len(s2):
```

```
        return False
```

```
    allowed = {frozenset(p) for p in pairs}
    return all(a == b or frozenset((a, b)) in allowed
               for a, b in zip(s1, s2))


# FOLLOW-UP 1: Similarity is transitive (a ~ b and b ~ c
# means a ~ c).
# Answer: Union-find over the words: union each pair, then
# two words are similar if they are equal or have the same
# root. Path compression keeps it near O(n + P).
def similar_transitive(s1, s2, pairs):
    # follow-up: a ~ b and b ~ c means a ~ c, so words are
    # grouped with union-find
    if len(s1) != len(s2):
        return False
    groups = DisjointSet()
    for a, b in pairs:
        groups.union(a, b)
    return all(a == b or groups.find(a) == groups.find(b)
               for a, b in zip(s1, s2))


# FOLLOW-UP 2: One sentence is the other with a sentence
# of words inserted somewhere (LeetCode 1813).
# Answer: Match the longest common prefix and the longest
# common suffix of the word lists; the shorter list must
# be covered by prefix plus suffix.
def similar_insertion(s1, s2):
    # follow-up (LeetCode 1813): one sentence is the other
    # with a block of words inserted somewhere. The
    # shorter
    # one must be covered by a common prefix plus a common
    # suffix of the two.
    if len(s1) > len(s2):
        s1, s2 = s2, s1
    head = 0
    while head < len(s1) and s1[head] == s2[head]:
        head += 1
    tail = 0
    room = len(s1) - head
    while tail < room and s1[-1 - tail] == s2[-1 - tail]:
        tail += 1
    return head + tail == len(s1)
```

```
# FOLLOW-UP 3: Case should not matter, or words can have
# synonyms with scores.
# Answer: Normalise with casefold before hashing. For
# scores, a graph with weighted edges and a threshold
# query turns it into reachability along strong edges.
def normalise(words):
    # follow-up: case does not matter
    return [w.casefold() for w in words]


if __name__ == "__main__":
    pairs = [("great", "good"), ("acting", "drama"),
             ("skills", "talent")]
    a = "You have great acting skills".split()
    b = "You have good drama talent".split()
    assert similar_direct(a, b, pairs)
    assert not similar_direct(a, b[:-1], pairs)
    assert not similar_direct(["a"], ["b"], [])
    assert similar_direct(["a"], ["a"], [])
    # chain: good ~ fine and fine ~ okay, so good ~ okay
    chain = [("good", "fine"), ("fine", "okay")]
    assert not similar_direct(["good"], ["okay"], chain)
    assert similar_transitive(["good"], ["okay"], chain)
    assert similar_insertion("My name is Haley".split(),
                             "My Haley".split())
    assert similar_insertion(["of"], ["A", "lot", "of"])
    eating = "Eating right now".split()
    assert similar_insertion(eating, ["Eating"])
    assert not similar_insertion(["a", "b"], ["c", "d"])
    assert normalise(["Great"]) == ["great"]
    import random
    for _ in range(300):
        words = "abcde"
        pr = [tuple(random.sample(words, 2))
              for _ in range(random.randint(0, 4))]
        x, y = random.choice(words), random.choice(words)
        want = reachable(x, y, pr)
        assert similar_transitive([x], [y], pr) == want
```

OFFICIAL PAGE TEXT

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples

```
sentence1 = ["Let's", "code", "in", "Python"]
sentence2 = ["Let's", "program", "in", "Python"]
similarPairs = [
    ("code", "program"),
]
output: true

sentence1 = ["I", "love", "to", "play", "football"]
sentence2 = ["I", "love", "playing", "soccer"]
similarPairs = [("play", "playing"), ("football", "soc
cer")]
output: false, different sentence lengths

sentence1 = ["Do", "you", "like", "coffee"]
sentence2 = ["Do", "you", "love", "coffee"]
similarPairs = [
    ("like", "enjoy"),
    ("coffee", "tea"),
]
output: false  # "like" is not similar to "love" based
 on the given pairs.

sentence1 = ["I", "really", "love", "leetcode", "and",
 "apples"]
sentence2 = ["I", "so", "like", "codesignal", "and", "
oranges"]
similarPairs = [
    ("very", "so"),
    ("love", "adore"),
    ("really", "very"),
    ("leetcode", "codesignal"),
```

```
    ("apples", "oranges"),
    ("like", "adore"),
]
output: true
```

OFFICIAL PAGE TEXT

The provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are connected if they are similar according to the provided pairs. Then, it performs a depth-first search (DFS) to check if each pair of corresponding words from the two sentences are connected in the graph. The Depth-First Search (DFS) algorithm is well-suited for this question as it efficiently explores the graph structure created from the similar word pairs. DFS is ideal for checking connectivity between words, which aligns with the task of determining if two words are similar based on the provided pairs. Its memory efficiency is beneficial for handling potentially large sets of sentences and word pairs. Additionally, DFS offers a simple implementation, either through recursion or using a stack, which aids in writing clean and understandable code. Its flexibility allows for easy modification to incorporate additional constraints or optimizations if needed. Therefore, DFS provides an elegant and effective solution for traversing the graph and determining sentence similarity. Remember to handle potential off-by-one errors when implementing this solution due to index-based iteration over the sentences. Time Complexity: Let n be the maximum number of words in the sentences, and m be the number of similar word pairs. Constructing the

graph takes O(m) time. Then, for each word pair, the DFS takes O(n) time in the worst case, resulting in a total time complexity of O(n + m). Space Complexity: In the worst case, the graph can contain O(m) edges. Additionally, in the DFS, the space complexity is O(n) due to the recursion stack or auxiliary data structures to track visited nodes. Therefore, the overall space complexity is O(n + m).Watch Simon, software engineer at Google, answer the question "determine if two sentences are similar."No answers contributed by the community yet.Related questionsDetermine if a number is a power of two.Determine if two words are sorted in lexicographic order.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start lea

rningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

HARD | PracHub | Technical Screen | 11 of 20

Find the Longest Straight Run of One Color

POSTED 2026-08-23 | 58 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 19.6 30 DAYS 0 PRIORITY 145/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix

Posted on PracHub: 2026-08-23 (updated 2026-09-10)

Original: https://prachub.com/coding-questions/find-the-longest-straight-run-of-one-color

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **`R`** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

QUESTION

The down-left diagonal from the top-right cell contains three **`G`** cells. No horizontal or vertical sequence has that length.

APPROACH

Per-cell DP over four directions (horizontal, vertical, diagonal, anti-diagonal): run[d][i][j] = run[d][prev]+1 if same color else 1. O(m*n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether diagonals count in both directions (yes, anti-diagonals too), whether cells must share an exact color value, and what to return for an empty grid. Confirm that a run cannot bend or skip a cell.

BRUTE FORCE TO SAY OUT LOUD

For every cell and each of the four directions, walk forward while the color matches. That is O(m * n * min(m, n)) and is a fine first answer to say out loud.

OPTIMAL IDEA

Let run[d][i][j] be the length of the same-color run that ends at (i, j) going in direction d. It is run[d][prev] + 1 if the previous cell in that direction has the same color, else 1. Four directions need only the previous row, so memory can be O(n). Answer is the maximum over everything. Time O(m * n).

DRY-RUN THESE CASES

A single cell (answer 1). A 1 x n row of one color. A grid where the longest run is on the anti-diagonal. A grid where every neighbor differs (answer 1).

LIKELY FOLLOW-UPS

Return the cells of the run, not just the length. Allow one wildcard cell. Make the run length at least K and count how many such runs exist.

PITFALLS

Forgetting the anti-diagonal needs the up-right previous cell, so it must be filled in the right scan order. Counting a bent path as a run.

HOW TO THINK ABOUT IT

A run in a fixed direction depends only on the cell before it in that direction, so each of the four directions is a one-line recurrence (like longest common suffix). Only four directions are needed because a line read backwards is the same line. Rolling only the previous row keeps memory at O(n).

Basis: Written; the same recurrence as LeetCode 562, Longest Line of Consecutive One in Matrix, with any color instead of 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 1
for r, c in all_cells(grid):
    for dr, dc in DIRECTIONS:  # 4 directions
        n = 1
        while same(grid, r, c,
                   r + n * dr, c + n * dc):
            n += 1
        best = max(best, n)
```

TIME	SPACE
O(m * n * min(m, n))	O(1)

WHY IT IS SLOW

A long run is re-walked from every cell inside it, O(m * n * min(m, n)). A table of the run length ending at each cell per direction is O(m * n).

OPTIMAL SOLUTION · DP per direction with a rolling row

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step back by its length.

FOLLOW-UP 2 Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

FOLLOW-UP 3 The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the Longest Straight Run of One
# Color
#
# Given a rectangular grid of colors, return the length of
# the longest consecutive sequence of cells with the same
# color along a horizontal, vertical, or diagonal straight
# line. A sequence cannot change direction or skip a cell.
# Both diagonal orientations count. The length is the
# number of cells in the sequence. ### Input - grid: an
# array of strings of equal length. Each character
# represents one cell's color. ### Output Return the
# maximum length as an integer. ### Constraints and Edge
# Cases - For this practice version, the grid contains at
# least one row and one column, with at most 1000000 cells
# in total. - Colors are uppercase English letters; equal
# letters mean equal colors. - A one-cell grid has answer
# 1. - Diagonal lines may run down-right or down-left.
# Reading the same line in reverse does not change its
# length. - A connected region that bends is not
# necessarily one valid sequence.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the run is, not only its length.
# 2. Require the run to be at least K and count how many
# such runs exist.
# 3. The grid is huge and arrives row by row.
#
# INTERVIEW NOTES
# A run in a fixed direction depends only on the cell
# before it in that direction, so each of the four
# directions is a one-line recurrence (like longest common
# suffix). Only four directions are needed because a line
# read backwards is the same line. Rolling only the
# previous row keeps memory at O(n).
#
# BRUTE FORCE (say it first): From every cell, walk in
# each of four directions
# For each start cell and direction, step while the color
# matches. Correct and short, but cells in a long run are
# re-walked from every start. Time O(m * n * min(m, n)).
# Space O(1).


# BRUTE FORCE (say it first): From every cell, walk in
# each of four directions | time O(m * n * min(m, n))
```

```
DIRS4 = [(0, 1), (1, 0), (1, 1), (1, -1)]


def longest_run_brute(grid):
    best = 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            for dr, dc in DIRS4:
                k = 0
                while (
                    0 <= r + dr * k < len(grid)
                    and 0 <= c + dc * k < len(grid[0])
                    and grid[r + dr * k][c + dc * k]
                    == grid[r][c]
                ):
                    k += 1
                best = max(best, k)
    return best


# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


def runs(grid):
    # yield (length, start, direction) for every maximal
    # run;
    # a run is walked only from its first cell, so the
    # total
    # work is linear in the number of cells
    rows, cols = len(grid), len(grid[0])
```

```
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                pr, pc = r - dr, c - dc
                before = 0 <= pr < rows and 0 <= pc < cols
                if before and grid[pr][pc] == grid[r][c]:
                    # not the first cell of its run
                    continue
                n, a, b = 0, r, c
                while 0 <= a < rows and 0 <= b < cols \
                        and grid[a][b] == grid[r][c]:
                    n += 1
                    a, b = a + dr, b + dc
                yield n, (r, c), (dr, dc)
```

```
# BASE QUESTION (optimal): DP per direction with a rolling
# row
# Idea: run[d][i][j] is the length of the same-color run
# ending at (i, j) in direction d. It is the previous
# cell's run plus one when the colors match, else 1.
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    return max(n for n, _, _ in runs(grid))


# FOLLOW-UP 1: Return where the run is, not only its
# length.
# Answer: Only start a walk at cells that begin a run (the
# previous cell in that direction has a different color);
# that keeps the total work linear, and you record the
# start cell and direction of the best run.
def longest_run_where(grid):
    # the length, the start cell and the direction
    if not grid or not grid[0]:
        return 0, None, None
    return max(runs(grid), key=lambda x: x[0])


# FOLLOW-UP 2: Require the run to be at least K and count
# how many such runs exist.
# Answer: Count a run once, at its start, when its length
# is at least K (use the begins-a-run test above).
# Counting at every cell would count a long run many
# times.
def count_runs_at_least(grid, k):
    # each run is counted once, at its start
    if not grid or not grid[0]:
        return 0
    return sum(1 for n, _, _ in runs(grid) if n >= k)


# FOLLOW-UP 3: The grid is huge and arrives row by row.
# Answer: The rolling-row version already needs only the
# previous row, so it works on a stream of rows with O(n)
# memory.
def longest_run_rows(row_stream):
    # rows arrive one by one: keep only the previous row's
```

```
    # runs
    best, prev = 0, None
    h = v = d = a = None
    for row in row_stream:
        n = len(row)
        nh, nv = [1] * n, [1] * n
        nd, na = [1] * n, [1] * n
        for c in range(n):
            if c and row[c] == row[c - 1]:
                nh[c] = nh[c - 1] + 1
            if prev is not None:
                if row[c] == prev[c]:
                    nv[c] = v[c] + 1
                if c and row[c] == prev[c - 1]:
                    nd[c] = d[c - 1] + 1
                if c + 1 < n and row[c] == prev[c + 1]:
                    na[c] = a[c + 1] + 1
            best = max(best, nh[c], nv[c], nd[c], na[c])
        prev, h, v, d, a = row, nh, nv, nd, na
    return best


if __name__ == "__main__":
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_run(g1) == 4
    assert longest_run(["RBG", "BGR", "GRB"]) == 3
    assert longest_run(["R"]) == 1
    assert longest_run([]) == 0
    length, start, direction = longest_run_where(g1)
    assert length == 4
    # runs of length at least 3 are counted once each
    assert count_runs_at_least(g1, 3) == 4
    assert longest_run_rows(iter(g1)) == 4
```

FUNCTION AND TEST CASES

Signature

```
def longestColorRun(grid: list[str]) -> int:
```

Sample call

```
longestColorRun(['RRRG', 'RRBB', 'GRRR', 'BGBR'])
```

Test cases

Input	Expected
(['RRRG', 'RRBB', 'GRRR', 'BGBR'],)	4
(['RBG', 'BGR', 'GRB'],)	3

ALSO REPORTED · 1 more

Longest Same-Color Grid Sequence | 1Point3Acres | phone-screen | asked 2026-09-08 | Single report

https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

ALSO REPORTED · 1 more

Preparation: Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

HARD | PracHub | Technical Screen | 12 of 20

Convert Valid Snake Case to Lower Camel Case

POSTED 2026-04-16 | 187 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 184/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy string manipulation)

Posted on PracHub: 2026-04-16 (updated 2026-07-18)

Original: https://prachub.com/coding-questions/convert-valid-snake-case-to-lower-camel-case

QUESTION

Convert Valid Snake Case to

Lower Camel Case Given a valid snake-case identifier, convert it to lower camel case. The input consists of one or more lowercase ASCII words separated by single underscores. There is no leading underscore, trailing underscore, or pair of adjacent underscores. The output keeps the first word lowercase, removes each underscore, and capitalizes the first letter of every later word. Implement:

```
def solve(name: str) -> str: ...
```

Constraints

- `1 <= len(name) <= 100_000`
- `name` matches `[a-z]+(?:_[a-z]+)*`

Examples

```
"foo_bar" -> "fooBar" "already" -> "already" "http_api_name" -> "httpApiName"
```

APPROACH

Split on '_', keep the first word lowercase, capitalize the first letter of the rest, join. O(n). The Hard tag on PracHub looks like a labeling quirk.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether input can have digits, uppercase, leading or trailing underscores, or double underscores. The problem states it is valid, so say you will rely on that and mention what you would add if not.

BRUTE FORCE TO SAY OUT LOUD

Loop over characters and capitalize the character after each underscore. Linear and simple.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Split on '_', keep the first word as is, capitalize the first letter of each other word and join. O(n). The Hard label on PracHub looks like a labeling quirk; do not over-engineer it.

DRY-RUN THESE CASES

A single word, two words, many words, a one-letter word.

LIKELY FOLLOW-UPS

Convert back from camel to snake, or handle acronyms.

PITFALLS

Capitalizing the first word. Using replace on every underscore without capitalizing.

HOW TO THINK ABOUT IT

The statement says the input is valid (single underscores, no edge underscores), so rely on that and say what you would add if it were not. The pattern is 'first word unchanged, capitalize the first letter of each later word'.

Basis: Written. The PracHub Hard label looks like a labeling quirk; this is an easy string task.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def to_camel(s):
    out, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            out += ch.upper() if up else ch
            up = False
    return out
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Already O(n); it is just longer than needed. Splitting on '_' and capitalising each word after the first is the short form.

OPTIMAL SOLUTION · Split on underscores and capitalize

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input may have leading, trailing or repeated underscores, or mixed case.

Drop empty parts after the split, lowercase the first word, capitalize the rest. Say what you do with digits and non-letters, and keep the rule in one place.

FOLLOW-UP 2 Convert back from camel case to snake case.

Insert an underscore before each uppercase letter and lowercase it. Acronyms such as HTTPServer need a smarter rule (split between a lowercase-to-uppercase edge and between an acronym and the next word).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Convert Valid Snake Case to Lower Camel
# Case
#
# # Convert Valid Snake Case to Lower Camel Case Given a
# valid snake-case identifier, convert it to lower camel
# case. The input consists of one or more lowercase ASCII
# words separated by single underscores. There is no
# leading underscore, trailing underscore, or pair of
# adjacent underscores. The output keeps the first word
# lowercase, removes each underscore, and capitalizes the
# first letter of every later word. Implement: python def
# solve(name: str) -> str: ... ## Constraints - 1 <=
# len(name) <= 100_000 - name matches [a-z]+(?:_[a-z]+)*
# ## Examples text "foo_bar" -> "fooBar" "already" ->
# "already" "http_api_name" -> "httpApiName"
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input may have leading, trailing or repeated
# underscores, or mixed case.
# 2. Convert back from camel case to snake case.
#
# INTERVIEW NOTES
# The statement says the input is valid (single
# underscores, no edge underscores), so rely on that and
# say what you would add if it were not. The pattern is
# 'first word unchanged, capitalize the first letter of
# each later word'.
#
# BRUTE FORCE (say it first): Walk the characters and
# capitalize after each underscore
# Keep a flag that is set after an underscore; when set,
# append the uppercase of the next letter. Linear and
# fine, just longer than the split-based answer. Time
# O(n). Space O(n).


import re


# BRUTE FORCE (say it first): Walk the characters and
# capitalize after each underscore | time O(n)
def snake_to_camel_brute(s):
    out, up = [], False
    for ch in s:
        if ch == "_":
```

```
            up = True
        else:
            out.append(ch.upper() if up else ch)
            up = False
    return "".join(out)


# BASE QUESTION (optimal): Split on underscores and
# capitalize
# Idea: Split on '_', keep the first word, and capitalize
# the first letter of each other word with a join. For
# valid input there are no empty parts.
def snake_to_camel(s):
    # foo_bar_baz -> fooBarBaz
    first, *rest = s.split("_")
    caps = [w[:1].upper() + w[1:] for w in rest]
    return first + "".join(caps)


# FOLLOW-UP 1: The input may have leading, trailing or
# repeated underscores, or mixed case.
# Answer: Drop empty parts after the split, lowercase the
# first word, capitalize the rest. Say what you do with
# digits and non-letters, and keep the rule in one place.
def snake_to_camel_tolerant(s):
    # leading, trailing or repeated underscores and mixed
    # case: drop empty parts, lowercase the first word
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    first = words[0].lower()
    rest = [w.capitalize() for w in words[1:]]
    return first + "".join(rest)


# FOLLOW-UP 2: Convert back from camel case to snake case.
# Answer: Insert an underscore before each uppercase
# letter and lowercase it. Acronyms such as HTTPServer
# need a smarter rule (split between a lowercase-to-
# uppercase edge and between an acronym and the next
# word).
def camel_to_snake(s):
    # fooBarBaz -> foo_bar_baz, and HTTPServer ->
    # http_server
    # 1) split an acronym from the next word: HTTPServer
```

```
    # -> HTTP_Server
    s = re.sub(r"([A-Z]+)([A-Z][a-z])", r"\1_\2", s)
    # 2) split a lower-case letter or digit from an upper-
    # case one
    s = re.sub(r"([a-z0-9])([A-Z])", r"\1_\2", s)
    return s.lower()


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    got = snake_to_camel("get_http_response")
    assert got == "getHttpResponse"
    # messy input
    messy = snake_to_camel_tolerant("_Foo__BAR_")
    assert messy == "fooBar"
    assert snake_to_camel_tolerant("___") == ""
    # and back again
    assert camel_to_snake("fooBarBaz") == "foo_bar_baz"
    assert camel_to_snake("HTTPServer") == "http_server"
    assert camel_to_snake("x") == "x"
    camel = snake_to_camel("my_var_2")
    assert camel_to_snake(camel) == "my_var2"
```

FUNCTION AND TEST CASES

Signature

```
def solve(name) -> str:
```

Sample call

```
solve('a')
```

Test cases

Input	Expected
`('a',)`	`'a'`
`('already',)`	`'already'`

ALSO REPORTED · 2 more

Define the scope and requirements for a snake_case to camelCase converter function. | Exponent

https://www.aced.io/questions/6328/define-scope-snake-case-camel-case-converter

Snake Case to Camel Case with Repository-Wide Refactor | 1Point3Acres | tech-screen | asked 2026-07-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

Preparation: Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Strings

HARD | PracHub | Technical Screen | 13 of 20

Find Dictionary Matches for a Jammed String

POSTED 2026-04-09 | 194 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 134/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: LC 809 Expressive Words (inferred from overview)

Posted on PracHub: 2026-04-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/find-dictionary-matches-for-a-jammed-string

QUESTION

Overview: This question evaluates string-processing and pattern-matching skills, specifically run-length grouping of consecutive characters and efficient dictionary lookup, within the Coding & Algorithms domain.

APPROACH

Run-length encode the query and every dictionary word; a word matches if run characters align and each run length is compatible. Pre-group dictionary by compressed signature in a hash map for fast lookup.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what jammed means exactly: a letter repeated more than it should be, and whether the repeat count must be at least the word's count or at least some threshold. Confirm case, empty strings and whether a match requires the same letter order and same number of runs.

BRUTE FORCE TO SAY OUT LOUD

For each dictionary word, compare it with the query run by run. O(W * L) per query.

OPTIMAL IDEA

Compress each word into runs of (letter, count). Use the letter sequence (without counts) as a hash key so only words with the same run letters are candidates, then compare the counts under the agreed rule. For repeated queries a trie over run letters keeps lookups proportional to the number of runs.

DRY-RUN THESE CASES

Query 'heeelllo' against 'hello' and 'helo'. A query with fewer runs than the word. A single-letter word. A word whose runs are longer than the query's.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Stretch only allowed for runs of three or more, return all matches in input order, handle a huge dictionary with a trie.

PITFALLS

Treating 'ee' to 'e' as a match without checking the rule. Comparing total length instead of per-run counts.

HOW TO THINK ABOUT IT

'Same letters, stretched' means compare run-length encodings, not characters. The letter order of the runs has to match exactly, so the sequence of distinct run letters is a hash key that cuts the dictionary down before any counts are compared.

Basis: LeetCode 809, Expressive Words. Assumed rule (confirm it): a query run may be longer than the word's run only when the query run has length at least 3; otherwise the lengths must be equal.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def matches(query, words):
    runs_q = runs(query)  # [(letter, count)]
    result = []
    for word in words:
        runs_w = runs(word)
        if compatible(runs_w, runs_q):
            result.append(word)
    return result
```

TIME	SPACE
O(W * L) per query	O(L)

WHY IT IS SLOW

Every query touches every word, O(W * L), even words whose letters cannot match. Grouping words by their run letters in a hash map checks only the right bucket.

OPTIMAL SOLUTION · Index the dictionary by its run letters

TIME	SPACE
O(L) per query plus the size of the matching bucket	O(total dictionary length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A different stretch rule: any run may be longer, no minimum.

Only the comparison changes. The index (grouping by run letters) is the same, so make the minimum stretch a parameter, as the code does with min_stretch.

FOLLOW-UP 2 Millions of dictionary words and many queries.

Keep the group index in memory and put a trie over the run-letter strings if the keys share long prefixes. For counts compare only the first words in a bucket that survive a cheap check such as total length. **Cost:** O(L) per query to build the run key plus the size of the bucket scanned; building the index is O(total dictionary length) once.

FOLLOW-UP 3 The jammed string may also be missing letters.

That becomes an edit-distance style problem on run encodings, usually solved with DP over (query run, word run). Ask for the exact rule before coding it.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Dictionary Matches for a Jammed
# String
#
# This question evaluates string-processing and pattern-
# matching skills, specifically run-length grouping of
# consecutive characters and efficient dictionary lookup,
# within the Coding & Algorithms domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A different stretch rule: any run may be longer, no
# minimum.
# 2. Millions of dictionary words and many queries.
# 3. The jammed string may also be missing letters.
#
# INTERVIEW NOTES
# 'Same letters, stretched' means compare run-length
# encodings, not characters. The letter order of the runs
# has to match exactly, so the sequence of distinct run
# letters is a hash key that cuts the dictionary down
# before any counts are compared.
#
# BRUTE FORCE (say it first): Compare the query with every
# dictionary word
# Encode the query once and compare it with each word's
# runs. It is correct; the cost is that every query
# touches every word, even words whose letters cannot
# match. Time O(W * L) per query. Space O(L).


from itertools import groupby


# HELPERS for the brute force
def _runs(s):
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return out


# BRUTE FORCE (say it first): Compare the query with every
# dictionary word | time O(W * L) per query
def _stretchy(query, word, min_stretch=3):
    a, b = _runs(query), _runs(word)
    if len(a) != len(b):
        return False
    return all(
        qa == wa
        and (
            qc == wc or (qc > wc and qc >= min_stretch)
        )
        for (qa, qc), (wa, wc) in zip(a, b)
    )


def jammed_matches_brute(words, query):
    return [w for w in words if _stretchy(query, w)]


# HELPERS for the base question
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]
    return [(ch, len(list(g))) for ch, g in groupby(word)]


# BASE QUESTION (optimal): Index the dictionary by its run
# letters
# Idea: Group words by the string of their distinct run
# letters ('hello' -> 'helo'). A query can only match
# words in its own group, so compare run counts only
# there. Build the index once and reuse it for many
# queries.
# FOLLOW-UP 1: A different stretch rule: any run may be
# longer, no minimum.
# Answer: Only the comparison changes. The index (grouping
# by run letters) is the same, so make the minimum stretch
# a parameter, as the code does with min_stretch.
class JammedIndex:
    """A query run may be longer than the word's run
    when it is at least min_stretch long."""

    def __init__(self, words):
        # words grouped by their run letters ("hello" ->
        # "helo")
        self.groups = {}
        for w in words:
            r = runs(w)
            letters = "".join(ch for ch, _ in r)
            self.groups.setdefault(letters, []).append(
                (w, [n for _, n in r]))

    # FOLLOW-UP 1: min_stretch=1: any run may be longer
    def matches(self, query, min_stretch=3):
        r = runs(query)
        letters = "".join(ch for ch, _ in r)
        counts = [n for _, n in r]
        found = []
        # only words with the same run letters can match
        group = self.groups.get(letters, [])
        for word, word_counts in group:
            if all(self.fits(q, w, min_stretch)
                   for q, w in zip(counts, word_counts)):
                found.append(word)
        return sorted(found)

    @staticmethod
    def fits(q, w, min_stretch):
        # the same length, or a stretch of at least
        # min_stretch
        return q == w or (q > w and q >= min_stretch)


if __name__ == "__main__":
    index = JammedIndex(["hello", "help", "hi", "hip"])
    # e stretched to 4 and o stretched to 3
```

```
    assert index.matches("heeeelloooo") == ["hello"]
    assert index.matches("hello") == ["hello"]
    # a stretch of only 2 is not enough
    assert index.matches("heello") == []
    assert index.matches("hiiii") == ["hi"]
    assert index.matches("zzz") == []
    # a smaller minimum accepts the short stretch
    short = index.matches("heello", min_stretch=2)
    assert short == ["hello"]
    # no minimum: any longer run is fine
    assert index.matches("hiip", min_stretch=1) == ["hip"]
    want = [("a", 2), ("b", 1), ("c", 3)]
    assert runs("aabccc") == want
```

FUNCTION AND TEST CASES

Signature

```
def solution(dictionary_words: list[str], jammed_string: s
tr) -> list[str]:
```

Sample call

```
solution(['help', 'hello', 'shell', 'hellp'], 'hellp')
```

Test cases

Input	Expected
(['help', 'hello', 'shell', 'hellp'], 'hellp')	['help', 'hellp']
(['hello', 'halo', 'heeello'], 'hheeellllo')	['hello', 'heeello']

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Recursion and backtracking

Site tags: recursion, simulation, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 14 of 20

Mahjong Winning Hand Detection (14 Tiles)

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 210/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

APPROACH

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

HOW TO THINK ABOUT IT

A winning hand is one pair plus four melds (a triplet of identical tiles or a run of three consecutive ranks in one suit). Try every tile as the pair, then repeatedly take the smallest remaining tile: it can only be covered by a triplet of itself or by a run that starts at it, so there are at most two choices at each step and the search is tiny for 14 tiles.

HOW TO THINK ABOUT IT

Basis: Written for this question; the pair-plus-melds check is the same backtracking as LeetCode 846, Hand of Straights, plus a triplet option.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(tiles):
    for t in set(tiles):
        if tiles.count(t) >= 2:
            rest = remove(tiles, [t, t])
            if all_melds(rest):
                return True
    return False
def all_melds(tiles):
    if not tiles:
        return True
    first = min(tiles)
    for meld in (triple(first), run(first)):
        if contains(tiles, meld):
            if all_melds(remove(tiles, meld)):
                return True
    return False
```

TIME	SPACE
roughly C(12,3) * C(9,3) ... roughly C(12,3) * C(9,3) * C(6,3) per pair	**O(1)**

WHY IT IS SLOW

Tries every set of three tiles at every step. The smallest remaining tile can only be in a triplet of itself or a run starting at it, so at most two choices remain.

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

TIME	SPACE
O(1) for 14 tiles (about 2^4 branches per pair)	**O(distinct tiles)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Mahjong Winning Hand Detection (14 Tiles)
#
# Mountain View onsite coding round 1: given a hand of 14
# tiles, decide whether it is a winning mahjong hand. The
# candidate must clarify the rule set (typically: 4 melds
# of three identical tiles or three consecutive ranks + 1
# pair). Input: array of 14 integers representing tile
# values (clarify suit / range during the interview). A
# winning hand consists of: One pair (two identical
# tiles), and Four melds, each being either: A triplet (3
# identical tiles), or A run (3 consecutive ranks in the
# same suit). Minority variant: A newer version instead
# specifies one pair plus two melds while retaining a
# 14-tile input. Because that accounts for only eight
# tiles, clarify the intended meld count and how the
# remaining six tiles should be handled before coding.
# Output: boolean.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A variant needs only two melds plus a pair (8 tiles
# of 14).
#
# INTERVIEW NOTES
# A winning hand is one pair plus four melds (a triplet of
# identical tiles or a run of three consecutive ranks in
# one suit). Try every tile as the pair, then repeatedly
# take the smallest remaining tile: it can only be covered
# by a triplet of itself or by a run that starts at it, so
# there are at most two choices at each step and the
# search is tiny for 14 tiles.
#
# BRUTE FORCE (say it first): Choose any three tiles for
# every meld
# For each pair, try every set of three remaining tiles
# that forms a meld and recurse. Correct and the oracle;
# it ignores that the smallest tile must be used first.
# Time roughly C(12,3) * C(9,3) * C(6,3) per pair. Space
# O(1).


import itertools
from collections import Counter


# BRUTE FORCE (say it first): Choose any three tiles for
# every meld | time roughly C(12,3) * C(9,3) * C(6,3) per
# pair
def mahjong_brute(hand):
    def is_meld(t):
        t = sorted(t)
        same = t[0] == t[1] == t[2]
        run = (
            t[0] // 10 == t[1] // 10 == t[2] // 10 < 3
            and t[1] == t[0] + 1
            and t[2] == t[0] + 2
        )
        return same or run

    def melds(tiles):
        if not tiles:
            return True
        for idx in itertools.combinations(
            range(len(tiles)), 3
        ):
            if is_meld(
                [tiles[i] for i in idx]
            ) and melds(
                [
                    t
                    for i, t in enumerate(tiles)
                    if i not in idx
                ]
            ):
                return True
        return False

    for i, j in itertools.combinations(
        range(len(hand)), 2
    ):
        if hand[i] == hand[j] and melds(
            [
                t
                for k, t in enumerate(hand)
                if k not in (i, j)
            ]
        ):
            return True
    return False
```

```
# HELPERS for the base question
def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2


def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
    return False


# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# Idea: Tiles are encoded as suit * 10 + rank for the
# numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and
# above for honours, which never form runs.
# FOLLOW-UP 1: A variant needs only two melds plus a pair
# (8 tiles of 14).
# Answer: Ask what happens to the other six tiles before
# coding: if they are free, check whether any 8 tiles form
# a pair and two melds with the same search over subsets
```

```
# of the 14.
def is_winning_hand(hand, melds=4):
```

```
    # FOLLOW-UP 1: melds=2 handles the 8-tile variant
    # one pair plus `melds` melds uses every tile
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 11, 12, 13,
           21, 22, 23, 31, 31]
    assert is_winning_hand(win)
    # 2, 3, 5 is not a run
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13,
            21, 22, 23, 31, 31]
    assert not is_winning_hand(lose)
    # honours only make triplets or the pair
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33,
               34, 34, 34]
    assert is_winning_hand(honours)
    # the smaller variant: a pair and two melds (8 tiles)
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    # wrong tile count is never a win
    assert not is_winning_hand(win[:13])
```

NOTES FROM CANDIDATES

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:

Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.

For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

PREPARATION

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Recursion and backtracking

MEDIUM | PracHub | Technical Screen | 15 of 20

Count Prefix-Matching Simple Paths in a Labeled Graph

POSTED 2026-08-09 | 72 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 0 | PRIORITY 83/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #79 Word Search

LeetCode match: No direct match; related LC 79 Word Search (inferred from overview)

Posted on PracHub: 2026-08-09 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/count-prefix-matching-simple-paths-in-a-labeled-graph

QUESTION

Overview: Count distinct simple graph paths whose labels match every nonempty prefix of a query word. Paths may start anywhere but cannot revisit a vertex, so the task rewards subset-state search and careful counting over a small labeled graph.

APPROACH

DFS/backtracking from every start vertex, extending a path only while the labels match the next query character and the vertex is unvisited (visited bitmask); count paths for every prefix length.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the graph is directed, whether a path is counted as a vertex sequence, whether paths of every prefix length are counted (the statement says every nonempty prefix), and the size (small, since the state is a visited subset).

BRUTE FORCE TO SAY OUT LOUD

Enumerate all simple paths and then test labels. Exponential and wasteful.

OPTIMAL IDEA

DFS with backtracking from every start vertex. Extend a path only if the next vertex is unvisited (a bitmask of visited vertices) and its label equals the next character of the query word. Every time the path length is L, count one for prefix length L. The pruning on labels keeps the search small. Time depends on the number of matching paths, bounded by n * d^(L-1).

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A query of length 1 (count of vertices with that label), a graph with a cycle (simple paths may not revisit), a label that appears nowhere, repeated letters in the query.

LIKELY FOLLOW-UPS

Longer queries (memoize on (vertex, mask, position) only helps if counting, not listing), or undirected edges.

PITFALLS

Counting a path that revisits a vertex. Counting an undirected path twice (once in each direction) without agreeing on it.

HOW TO THINK ABOUT IT

A path is only worth extending while its labels still match the query, so prune on the label before going deeper. Distinctness is tracked with a visited set (a bitmask for small graphs). Count at every depth, since each prefix length has its own answer.

Basis: Written; backtracking on a small labelled graph. Assumed rules (confirm them): directed edges; a path is a sequence of distinct vertices; for each L from 1 to the length of the query, count the paths of L vertices whose labels spell the first L letters of the query.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
def dfs(path):
    global count
    if len(path) <= len(query):
        if matches_prefix(path):
            count += 1
    for nxt in graph[path[-1]]:
        if nxt not in path:
            dfs(path + [nxt])
for v in vertices:
    dfs([v])
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

It explores paths whose first label already disagrees with the query. Extending a path only when the next label matches the next query letter prunes those branches.

OPTIMAL SOLUTION · DFS with label pruning and a visited bitmask

TIME	SPACE
O(number of matching paths * degree)	O(path)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

FOLLOW-UP 2 Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

FOLLOW-UP 3 The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Prefix-Matching Simple Paths in a
# Labeled Graph
#
# Count distinct simple graph paths whose labels match
# every nonempty prefix of a query word. Paths may start
# anywhere but cannot revisit a vertex, so the task
# rewards subset-state search and careful counting over a
# small labeled graph.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges are undirected.
# 2. Count only the paths that spell the whole query.
# 3. The graph has up to 100,000 vertices.
#
# INTERVIEW NOTES
# A path is only worth extending while its labels still
# match the query, so prune on the label before going
# deeper. Distinctness is tracked with a visited set (a
# bitmask for small graphs). Count at every depth, since
# each prefix length has its own answer.
#
# BRUTE FORCE (say it first): Enumerate all simple paths,
# then test the labels
# List every simple path of every length and compare its
# labels with each prefix of the query. Correct and the
# oracle; it explores paths whose first label already
# disagrees. Time exponential. Space O(path).


from collections import defaultdict


# BRUTE FORCE (say it first): Enumerate all simple paths,
# then test the labels | time exponential
def prefix_paths_brute(labels, edges, query):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    counts = [0] * len(query)

    def go(path):
        word = "".join(labels[v] for v in path)
        if (
            len(path) <= len(query)
            and word == query[: len(path)]
        ):
            counts[len(path) - 1] += 1
        if len(path) < len(query):
            for w in adj[path[-1]]:
                if w not in path:
                    go(path + [w])

    for s in range(len(labels)):
        go([s])
    return counts


# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
# Idea: Start from every vertex whose label equals the
# first query letter. Extend to an unvisited neighbour
# only if its label equals the next query letter, adding
# one to the count for the current length at each step.
def prefix_paths(labels, edges, query, undirected=False):
    # counts[d]: simple paths whose first d + 1 labels
    # spell
    # the first d + 1 letters of the query
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        # FOLLOW-UP 1: undirected=True adds both
        # directions
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            # a simple path never revisits a vertex
            if mask >> w & 1:
                continue
            if labels[w] == query[depth + 1]:
                go(w, depth + 1, mask | 1 << w)

    # a path may start at any vertex with the first letter
    for s in range(len(labels)):
        if query and labels[s] == query[0]:
            go(s, 0, 1 << s)
    return counts


# VARIANT: walks may revisit vertices
def prefix_walks(labels, edges, query):
    # walks may revisit vertices: a DP over (depth,
    # vertex)
    # costs O(L * E) and works for very large graphs
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    ways = {v: 1 for v in range(len(labels))
            if query and labels[v] == query[0]}
    counts = [sum(ways.values())] if query else []
    for ch in query[1:]:
        nxt = defaultdict(int)
        for u, n in ways.items():
            for w in adj[u]:
```

```
                if labels[w] == ch:
                    nxt[w] += n
        ways = nxt
        counts.append(sum(ways.values()))
    return counts


# FOLLOW-UP 2: Count only the paths that spell the whole
# query.
# Answer: Return counts[-1] instead of the list. For long
# queries on bigger graphs, memoise on (vertex, depth) if
# repeated vertices are allowed (walks, not simple paths);
# for simple paths the visited set prevents memoisation.
def whole_query_paths(labels, edges, query,
                      undirected=False):
    # only the paths that spell the entire query
    both = undirected
    counts = prefix_paths(labels, edges, query, both)
    return counts[-1] if counts else 0


if __name__ == "__main__":
    labels = ["a", "b", "a", "b"]
    edges = [(0, 1), (1, 2), (2, 3), (3, 0)]
    # "aba": directed square a>b>a>b>...
    assert prefix_paths(labels, edges, "aba") == [2, 2, 2]
```

```
    assert whole_query_paths(labels, edges, "aba") == 2
    # undirected: every edge works both ways
    und = prefix_paths(labels, edges, "ab", True)
    assert und == [2, 4]
    # a walk may reuse vertices, a simple path may not
    walks = prefix_walks(labels, edges, "abab")
    assert walks == [2, 2, 2, 2]
    five = prefix_paths(labels, edges, "ababa")
    assert five == [2, 2, 2, 2, 0]
    assert prefix_paths(labels, edges, "") == []
```

FUNCTION AND TEST CASES

Signature

```
def count_prefix_paths(labels: str, edges: list[list[int]]
, word: str) -> list[int]:
```

Sample call

```
count_prefix_paths('roteoreote', [[0, 1], [0, 2], [0, 3],
[1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')
```

Test cases

Input	Expected
`('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')`	`[2, 2, 1, 1, 0]`
`('a', [], 'aa')`	`[1, 0]`

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Recursion and backtracking

MEDIUM | IN APP LC 46 | Exponent | 16 of 20

Permutations

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 49.8 30 DAYS 0 PRIORITY 70/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: your app's interview approach (LeetCode #46)

LeetCode match: LC 46 Permutations

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +25 interviews

Original: https://www.aced.io/questions/2212/leetcode-medium-permutations-dp

QUESTION

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example Input: nums = [1, 2, 3] Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

APPROACH

Backtracking with a used[] array (or swap-based in place). n! outputs.

HOW TO THINK ABOUT IT

The output has n! items, so no algorithm can beat O(n * n!); the question is only whether you generate them without waste. Build a permutation one position at a time, marking which elements are used, and undo the choice when you come back (backtracking). Every follow-up (duplicates, k-th, next) is a variation on 'what are the legal choices at this position'.

Basis: LeetCode 46 (Permutations), 47 (with duplicates), 31 (Next Permutation) and 60 (Permutation Sequence).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    result = []
    for seq in product(nums, repeat=len(nums)):
        if len(set(seq)) == len(nums):
            result.append(list(seq))
    return result
```

TIME	SPACE
O(n^n * n)	O(n!)

WHY IT IS SLOW

BRUTE FORCE

n^n sequences when only n! are valid, so almost all the work is wasted. Backtracking with a used marker only builds valid prefixes, O(n * n!).

OPTIMAL SOLUTION · Backtracking with a used array

TIME	SPACE
O(n * n!)	O(n) beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input has duplicates (Permutations II).

Sort first, and skip a value that equals the previous one unless the previous one is already used in the current prefix. That forces equal values to appear in a fixed order and removes duplicates without a set.

FOLLOW-UP 2 Return the next lexicographic permutation in place.

Find the last position i with a[i] < a[i + 1], swap a[i] with the smallest larger value to its right, then reverse the suffix. O(n), O(1) space. Repeat to enumerate in order.

FOLLOW-UP 3 Return only the k-th permutation of 1..n.

Use the factorial number system: the first digit is index (k - 1) // (n - 1)! of the remaining digits, then continue with the remainder. O(n^2), no enumeration.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Permutations
#
# Given an array of distinct integers, return all possible
# permutations of the array. A permutation of an array is
# a rearrangement of its elements. Example Input: nums =
# [1, 2, 3] Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2,
# 3, 1], [3, 1, 2], [3, 2, 1]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input has duplicates (Permutations II).
# 2. Return the next lexicographic permutation in place.
# 3. Return only the k-th permutation of 1..n.
#
# INTERVIEW NOTES
# The output has n! items, so no algorithm can beat O(n *
# n!); the question is only whether you generate them
# without waste. Build a permutation one position at a
# time, marking which elements are used, and undo the
# choice when you come back (backtracking). Every follow-
# up (duplicates, k-th, next) is a variation on 'what are
# the legal choices at this position'.
```

```
#
# BRUTE FORCE (say it first): Generate all n^n sequences
# and keep the valid ones
# Enumerate every sequence of length n over the values and
# filter those with all values distinct. It shows the idea
# but wastes almost all of its work (n^n versus n!). Time
# O(n^n * n). Space O(n!).


import itertools
from math import factorial


# BRUTE FORCE (say it first): Generate all n^n sequences
# and keep the valid ones | time O(n^n * n)
def permute_brute(nums):
    return [
        list(seq)
        for seq in itertools.product(
            nums, repeat=len(nums)
        )
        if len(set(seq)) == len(nums)
    ]
```

```
# BASE QUESTION (optimal): Backtracking with a used array
# Idea: Fill positions left to right. At each position try
# every unused value, mark it, recurse, and unmark. Only
# valid prefixes are ever extended, so every step leads to
# a permutation.
def permute(nums):
    # backtracking: choose an unused element, recurse,
    # undo
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if not used[i]:
                used[i] = True
                cur.append(x)
                go()
                cur.pop()
                used[i] = False

    go()
    return out


# FOLLOW-UP 1: The input has duplicates (Permutations II).
# Answer: Sort first, and skip a value that equals the
# previous one unless the previous one is already used in
# the current prefix. That forces equal values to appear
# in a fixed order and removes duplicates without a set.
def permute_unique(nums):
    # duplicates: sort, and skip a value equal to the
    # previous
    # one unless that one is already in the current prefix
    nums = sorted(nums)
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if used[i]:
```

```
                continue
            same = i and nums[i] == nums[i - 1]
            if same and not used[i - 1]:
                continue
            used[i] = True
            cur.append(x)
            go()
            cur.pop()
            used[i] = False

    go()
    return out


# FOLLOW-UP 2: Return the next lexicographic permutation
# in place.
# Answer: Find the last position i with a[i] < a[i + 1],
# swap a[i] with the smallest larger value to its right,
# then reverse the suffix. O(n), O(1) space. Repeat to
# enumerate in order.
def next_permutation(a):
    # in place: O(n) time, O(1) space; False when it was
    # the last
    i = len(a) - 2
    # the last position that can still increase
    while i >= 0 and a[i] >= a[i + 1]:
        i -= 1
    if i < 0:
        a.reverse()
        return False
    j = len(a) - 1
    while a[j] <= a[i]:
        j -= 1
    a[i], a[j] = a[j], a[i]
    # the suffix is decreasing: make it the smallest
    a[i + 1:] = reversed(a[i + 1:])
    return True


# FOLLOW-UP 3: Return only the k-th permutation of 1..n.
# Answer: Use the factorial number system: the first digit
# is index (k - 1) // (n - 1)! of the remaining digits,
# then continue with the remainder. O(n^2), no
# enumeration.
def kth_permutation(n, k):
```

```
    # the k-th (1-based) permutation of 1..n without
    # listing
    digits, out, k = list(range(1, n + 1)), [], k - 1
    for i in range(n, 0, -1):
        block = factorial(i - 1)
        out.append(digits.pop(k // block))
        k %= block
    return out


if __name__ == "__main__":
    assert permute([1, 2, 3]) == [
        [1, 2, 3], [1, 3, 2], [2, 1, 3],
        [2, 3, 1], [3, 1, 2], [3, 2, 1]]
    assert permute([]) == [[]]
    assert permute_unique([1, 1, 2]) == [
        [1, 1, 2], [1, 2, 1], [2, 1, 1]]
    a = [1, 2, 3]
    assert next_permutation(a) and a == [1, 3, 2]
    last = [3, 2, 1]
    assert not next_permutation(last)
    assert last == [1, 2, 3]
    # walking next_permutation lists every order
    seen, p = [], [1, 2, 3]
    while True:
        seen.append(p[:])
        if not next_permutation(p):
            break
    assert seen == permute([1, 2, 3])
    assert kth_permutation(3, 3) == [2, 1, 3]
```

```
assert kth_permutation(4, 9) == [2, 3, 1, 4]
```

OFFICIAL PAGE TEXT

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example

```
Input: nums = [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [
3, 1, 2], [3, 2, 1]]
```

Our solution generates all permutations of a given array by utilizing a recursive backtracking approach. The idea is to explore every possible way of arranging the elements by swapping them at each recursive call. Here's a breakdown of how the algorithm works: We define a helper function (backtrack) that recursively builds the permutations. This function takes two parameters: start (the index from where we start permuting) and end (the length of the array). The base case occurs when start == end, meaning that a complete permutation has been formed. We add a copy of the current permutation (stored in nums) to the result. In each recursive call, we iterate over the elements starting from the start index. For each element at position i, we: Swap the elements at start and i to bring a new element into the current position. Recursively call backtrack with start + 1, moving to the next index to fill. After the recursive call, we swap back the elements to restore the original order and continue exploring other possibilities. This process continues until all permutations are generated. The swapping ensures that we explore every possible combination of the elements. Time Complexity: The time complexity of this solution is O(n * n!), where n is the number of elements in the array. The reason for this is that generating each permutation takes O(n) time, and there are n! permutations. Space Complexity: The space

complexity is O(n!) due to the storage of all permutations in the result list. Additionally, the recursion stack uses O(n) space for the recursive calls, but this is dominated by the result storage.No answers contributed by the community yet.

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Recursion and backtracking

MEDIUM | IN APP LC 22 | Exponent | 17 of 20

★ TOP 75: rank 19 in the Medium and Hard priority list

★ Generate Parentheses

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 61.6 | 30 DAYS 46.9 | PRIORITY 13/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: your app's interview approach (LeetCode #22)

LeetCode match: LC 22 Generate Parentheses

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age) | +36 interviews

Original: https://www.aced.io/questions/1594/generate-parentheses

QUESTION

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples n = 1 output: ["()"] n = 2 output: ["(())", "()()"] n = 3 output: ["((()))", "(()())", "(())()", "()(())", "()()()"] n = 4 output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(())(())", "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and

closing parentheses that can still be added. It also

APPROACH

Backtracking: add '(' while open < n and ')' while close < open. Catalan-many outputs.

HOW TO THINK ABOUT IT

Build the string left to right and only make moves that can still lead to a valid result: you may add '(' while fewer than n have been opened, and ')' only while there are more opens than closes. Because invalid prefixes are never extended, every leaf is a valid answer, and the count is the Catalan number.

Basis: LeetCode 22, Generate Parentheses: all well-formed strings with n pairs.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    result = []
    for seq in product("()", repeat=2 * n):
        if is_balanced(seq):
            result.append("".join(seq))
    return result
```

TIME	SPACE
O(2^(2n) * n)	O(2^(2n))

WHY IT IS SLOW

Builds exponentially many invalid strings. Backtracking only adds '(' while fewer than n are used and ')' while fewer than the opens, so it builds only valid prefixes (the Catalan number of results).

OPTIMAL SOLUTION · Backtracking with open and close counters

TIME	SPACE
O(4^n / sqrt(n))	O(n) beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the k-th string, or just the number of strings.

The count is the Catalan number C(n) = (2n choose n) / (n + 1). For the k-th string count the completions that start with '(' at each step and skip them if k is larger (unranking), using a small DP over (opens left, balance).

FOLLOW-UP 2 Several bracket types, such as () [] {}.

Multiply the choices: at each position you may open any of the types (until its count is used) and may close only the type on top of a stack. The pruning rule (never close more than you have opened) becomes 'close only the most recent unclosed bracket'.

FOLLOW-UP 3 Avoid recursion.

Use an explicit stack of (string so far, opens, closes) states or build the strings by length with a queue. The pruning conditions stay the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Generate Parentheses
#
# Generate all combinations of well-formed parentheses. A
# parenthesis combination is well-formed if each opening
# parenthesis "(" is closed by a matching closing
# parenthesis ")" in the correct order. For example, "()",
# "()()", and "((()))" are all combinations of well-formed
# parentheses, while ")(", "((" and "(()" are not.
# Examples n = 1 output: ["()"] n = 2 output: ["(())",
# "()()"] n = 3 output: ["((()))", "(()())", "(())()",
# "()(())", "()()()"] n = 4 output should contain:
# ["(((())))", "((()()))", "((())())", "((()))()",
# "(()(()))", "(()()())", "(())(())", "(())()()",
# "()((()))", "()(()())", "()(())()", "()()(())",
# "()()()()"]The problem is to generate all
# combinations of well-formed parentheses given n pairs of
# parentheses. A combination is well-formed if every
# opening parenthesis has a corresponding closing
# parenthesis and they are properly nested.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the k-th string, or just the number of
# strings.
# 2. Several bracket types, such as () [] {}.
# 3. Avoid recursion.
#
# INTERVIEW NOTES
# Build the string left to right and only make moves that
# can still lead to a valid result: you may add '(' while
# fewer than n have been opened, and ')' only while there
# are more opens than closes. Because invalid prefixes are
# never extended, every leaf is a valid answer, and the
# count is the Catalan number.
#
# BRUTE FORCE (say it first): Generate all 2^(2n) strings
# and keep the balanced ones
# Enumerate every sequence of n opens and n closes and
# test balance. Correct and the oracle; it builds
# exponentially many invalid strings that backtracking
# never creates. Time O(2^(2n) * n). Space O(2^(2n)).


import itertools
from functools import lru_cache
from math import comb
```

```
# BRUTE FORCE (say it first): Generate all 2^(2n) strings
# and keep the balanced ones | time O(2^(2n) * n)
def _valid(s, t, k):
    return (
        len(s) == len(t)
        and sorted(s) == sorted(t)
        and all(
            t[i] != t[j]
            for i in range(len(t))
            for j in range(i + 1, min(len(t), i + k))
        )
    )


def parens_brute(n):
    return sorted(
        "".join(p)
        for p in set(
            itertools.permutations("(" * n + ")" * n)
        )
        if _valid(p)
    )


# HELPERS for follow-up 1
@lru_cache(maxsize=None)
def completions(opens_left, balance):
    # how many ways to finish a string that still has
    # opens_left "(" to place and balance unclosed ones
    if opens_left == 0:
        return 1
    total = completions(opens_left - 1, balance + 1)
    if balance:
        total += completions(opens_left, balance - 1)
    return total


# BASE QUESTION (optimal): Backtracking with open and
# close counters
# Idea: Track how many '(' and ')' have been placed. Add
# '(' if open < n; add ')' if close < open. When the
# string reaches length 2n it is valid.
def generate(n):
```

```
    # build the string one character at a time. Adding "("
    # is allowed while opens < n; adding ")" only while
    # closes < opens: that rule keeps every string valid.
    out = []

    def build(text, opens, closes):
        if len(text) == 2 * n:
            out.append(text)
            return
        if opens < n:
            build(text + "(", opens + 1, closes)
        if closes < opens:
            build(text + ")", opens, closes + 1)

    build("", 0, 0)
    return out


# FOLLOW-UP 1: Return only the k-th string, or just the
# number of strings.
# Answer: The count is the Catalan number C(n) = (2n
# choose n) / (n + 1). For the k-th string count the
# completions that start with '(' at each step and skip
# them if k is larger (unranking), using a small DP over
# (opens left, balance).
def catalan(n):
    # follow-up: just the NUMBER of strings
    return comb(2 * n, n) // (n + 1)


# FOLLOW-UP 1: Return only the k-th string, or just the
# number of strings.
# Answer: The count is the Catalan number C(n) = (2n
# choose n) / (n + 1). For the k-th string count the
# completions that start with '(' at each step and skip
# them if k is larger (unranking), using a small DP over
# (opens left, balance).
def kth(n, k):
    # follow-up: the k-th string (1-based, sorted order,
```

```
    # "("
    # before ")") without listing them: at each step count
    # the
    # strings that continue with "(" and skip them if
    # needed
    if not 1 <= k <= catalan(n):
        raise ValueError("k is out of range")
    text, opens_left, balance = "", n, 0
    while len(text) < 2 * n:
        ways = 0
        if opens_left:
            ways = completions(opens_left - 1,
                               balance + 1)
        if opens_left and k <= ways:
            text += "("
            opens_left -= 1
            balance += 1
        else:
            if opens_left:
                k -= ways
            text += ")"
            balance -= 1
    return text


# FOLLOW-UP 2: Several bracket types, such as () [] {}.
# Answer: Multiply the choices: at each position you may
# open any of the types (until its count is used) and may
# close only the type on top of a stack.
def generate_multi(n, kinds="()[]{}"):
    # follow-up: several bracket types. Any type can open;
    # only the type on top of the stack can close.
    pairs = [kinds[i:i + 2]
             for i in range(0, len(kinds), 2)]
    out = []

    def build(text, stack, opened):
        if len(text) == 2 * n:
            out.append(text)
            return
        if opened < n:
            for open_ch, close_ch in pairs:
                build(text + open_ch, stack + [close_ch],
                      opened + 1)
        if stack:
            build(text + stack[-1], stack[:-1], opened)

    build("", [], 0)
    return out
```

```
# FOLLOW-UP 3: Avoid recursion.
# Answer: Use an explicit stack of (string so far, opens,
# closes) states or build the strings by length with a
# queue. The pruning conditions stay the same.
def generate_iter(n):
    # follow-up: no recursion. The same states on a stack;
    # ")" is pushed first so "(" is handled first and the
    # strings come out in sorted order.
    out, stack = [], [("", 0, 0)]
    while stack:
        text, opens, closes = stack.pop()
        if len(text) == 2 * n:
            out.append(text)
            continue
        if closes < opens:
            stack.append((text + ")", opens, closes + 1))
        if opens < n:
            stack.append((text + "(", opens + 1, closes))
    return out
```

```
if __name__ == "__main__":
    assert generate(1) == ["()"]
    assert generate(2) == ["(())", "()()"]
    assert sorted(generate(3)) == [
        "((()))", "(()())", "(())()", "()(())", "()()()"]
    assert generate(0) == [""]
    for n in range(0, 8):
        listed = generate(n)
        assert len(listed) == catalan(n)
        assert generate_iter(n) == sorted(listed)
    for n in range(1, 7):
        every = sorted(generate(n))
        for k, text in enumerate(every, 1):
            assert kth(n, k) == text
    assert catalan(3) == 5 and catalan(10) == 16796
    # 3 types: Catalan(n) shapes times 3 choices per pair
    for n in range(1, 5):
        got = generate_multi(n, "()[]{}")
        assert len(got) == catalan(n) * 3 ** n
        assert len(set(got)) == len(got)
    two = sorted(generate_multi(1, "()[]"))
    assert two == ["()", "[]"]
```

OFFICIAL PAGE TEXT

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples

```
n = 1
output: ["()"]

n = 2
output: ["(())", "()()"]

n = 3
output: ["((()))", "(()())", "(())()", "()(())", "()()
()"]

n = 4
output should contain: ["(((())))", "((()()))", "((()))
()()", "((()))()", "(()(()))", "(()()())", "(()())()",
"(())(())", "(())()()", "()((()))", "()(()())", "()(()
)()", "()()(())", "()()()()"]
```

OFFICIAL PAGE TEXT

The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also maintains the current combination of the string being formed. Base Case: If no more parentheses are remaining to add (both remaining_open and remaining_close are zero), the current combination is a valid sequence and is added to the result list. Recursive Case: Add an opening parenthesis (if there are any remaining and recursively call the function with one less opening parenthesis. Add a closing parenthesis) if the number of remaining closing parentheses is greater than the remaining opening

parentheses and recursively call the function with one less closing parenthesis. Pruning Invalid Paths: If at any point the number of closing parentheses left is less than the number of opening parentheses left, it means the current sequence cannot be completed to a valid sequence, so we stop exploring that path. Implementation Steps Initialize: Create a result list to store all valid sequences. Call the recursive helper function with initial values (n opening and n closing parentheses). Recursive Helper Function: Track the number of remaining opening and closing parentheses. Track the current combination being formed. Append valid combinations to the result list when the base case is reached. Backtrack: After adding an opening or closing parenthesis, remove it (backtrack) before trying the next possibility. This ensures that we explore all possible valid combinations. Example Let's consider n = 3: Start with an empty combination "", 3 opening, and 3 closing parentheses. Add an opening parenthesis: "(", 2 opening, and 3 closing. Continue adding opening parentheses until you can no longer add without violating the rules. Then add closing parentheses and ensure that at no point the closing parentheses exceed the opening ones. Once you form a valid combination like "((()))", add it to the result list. Backtrack and explore other combinations. Solution Time Complexity: The time

OFFICIAL PAGE TEXT

complexity of this solution is O(22n)O(2^{2n})O(22n). This is because each position in the string can either be an opening or closing parenthesis, and we need to generate all possible sequences of length 2n2n2n. Space Complexity: The space complexity of the solution is O(2n)O(2n)O(2n) which simplifies to O(n)O(n)O(n). This is because: The recursion depth is 2n2n2n in the worst case (each recursive call adds either an opening or a closing parenthesis). The combination string being formed is of length 2n2n2n. In this mock interview video, Aman (Google SWE) answers the interview question, "Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses."No answers contributed by the community yet.

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Recursion and backtracking

MEDIUM | IN APP LC 494 | Exponent | 18 of 20

Explain how to find a target sum in an array.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 41.1 | 30 DAYS 0 | PRIORITY 89/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: your app's interview approach (LeetCode #494)

LeetCode match: LC 494 Target Sum

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | 3 interviews

Original: https://www.aced.io/questions/1439/find-target-sum-array

QUESTION

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples nums = [1, 1, 1, 1, 1] targetSum = 3 output: 5 nums = [1, 2, 1] targetSum = 2 output: 2 nums = [1, 2, 7, 1, 5, 3] targetSum = 8 output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to reach the target sum.)The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of

symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\te

APPROACH

Count assignments: DP on sums (subset-sum transform: P = (total + target) / 2), or memoized DFS on (index, sum). O(n * sum).

HOW TO THINK ABOUT IT

Put a + in front of the numbers in a set P and a - in front of the rest N. Then sum(P) - sum(N) = S and sum(P) + sum(N) = total, so sum(P) = (total + S) / 2. The problem becomes 'count the subsets with this sum', a one-dimensional knapsack count. If (total + S) is odd or negative the answer is 0.

Basis: LeetCode 494, Target Sum: assign + or - to every number and count the assignments that add up to S (the data matches this reading; a Two Sum reading is given as a follow-up).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def go(i, total):
    if i == len(nums):
        return 1 if total == S else 0
    plus = go(i + 1, total + nums[i])
    minus = go(i + 1, total - nums[i])
    return plus + minus
answer = go(0, 0)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two branches per number: O(2^n). The numbers with a + sign form a subset with sum (total + S) / 2, so a subset-sum counting table is O(n * sum).

OPTIMAL SOLUTION · Subset-sum counting: choose P with sum (total + S) / 2

TIME	SPACE
O(n * sum)	O(sum)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The question was really 'find two numbers that add up to the target' (Two Sum).

One pass with a dictionary from value to index: for each x look for target - x among earlier values. O(n) time, O(n) space; with a sorted array use two pointers in O(1) space.

FOLLOW-UP 2 Only the existence of a subset with the target sum.

The same subset-sum table with booleans (or a bitset: reach |= reach << x) answers existence in O(n * sum / word size).

FOLLOW-UP 3 Negative numbers or a huge sum.

With negatives shift all values to be non-negative, or use a dictionary from reachable sum to count (memoised DFS on (index, sum)); with a huge sum the dictionary is only as large as the number of distinct reachable sums. **Cost:** O(n * R) with a dictionary over R reachable sums; shifting for negatives adds O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Explain how to find a target sum in an
# array.
#
# Determine the Number of Ways to Assign Symbols to Reach
# a Target Sum. You are given a list of non-negative
# integers, a1, a2, ..., an, and a target sum S. Your task
# is to assign each integer in the list a symbol + or -
# and calculate how many different ways there are to
# arrange these symbols such that the sum of the integers
# equals S after applying the assigned symbols. For
# example, if you have the list [1, 1, 1, 1, 1] and the
# target sum 3, one way to reach the target sum is to
# assign + to three 1s and - to two 1s, like so: +1 +1 +1
# +1 -1. Another way could be +1 -1 +1 +1 +1. You need to
# return the total number of unique arrangements that meet
# the target sum S.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The question was really 'find two numbers that add up
# to the target' (Two Sum).
# 2. Only the existence of a subset with the target sum.
# 3. Negative numbers or a huge sum.
#
# INTERVIEW NOTES
# Put a + in front of the numbers in a set P and a - in
# front of the rest N. Then sum(P) - sum(N) = S and sum(P)
# + sum(N) = total, so sum(P) = (total + S) / 2. The
# problem becomes 'count the subsets with this sum', a
# one-dimensional knapsack count. If (total + S) is odd or
# negative the answer is 0.
#
# BRUTE FORCE (say it first): Try every combination of
# signs
# Recursively choose + or - for each number and count the
# ways to land on S. Exact and the oracle; exponential.
# Time O(2^n). Space O(n).


from collections import Counter


# BRUTE FORCE (say it first): Try every combination of
# signs | time O(2^n)
def target_sum_brute(nums, S):
    def go(i, total):
```

```
        if i == len(nums):
            return int(total == S)
        return go(i + 1, total + nums[i]) + go(
            i + 1, total - nums[i]
        )

    return go(0, 0)
```

```
# BASE QUESTION (optimal): Subset-sum counting: choose P
# with sum (total + S) / 2
# Idea: Let need = (total + S) / 2 (return 0 if it is not
# a non-negative integer). ways[s] counts the subsets with
# sum s; for each number x update ways from high to low:
# ways[s] += ways[s - x]. The answer is ways[need].
def count_signs(nums, target):
    # +/- signs reaching `target` equal a subset with sum
    # (total + target) / 2 that gets the plus signs. So
    # the
    # question becomes: how many subsets have that sum?
    total = sum(nums)
    if abs(target) > total or (total + target) % 2:
        return -1
    goal = (total + target) // 2
    ways = [1] + [0] * goal
    for x in nums:
        # go downward so every number is used at most once
        for s in range(goal, x - 1, -1):
            ways[s] += ways[s - x]
    return ways[goal]


# FOLLOW-UP 1: The question was really 'find two numbers
# that add up to the target' (Two Sum).
# Answer: One pass with a dictionary from value to index:
# for each x look for target - x among earlier values.
# O(n) time, O(n) space; with a sorted array use two
# pointers in O(1) space.
def two_sum(nums, target):
    # the question may have meant Two Sum: one pass, a
    # dictionary from value to index of earlier numbers
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return seen[target - x], i
```

```
        seen[x] = i
    return None


# FOLLOW-UP 1: The question was really 'find two numbers
# that add up to the target' (Two Sum).
# Answer: One pass with a dictionary from value to index:
# for each x look for target - x among earlier values.
# O(n) time, O(n) space; with a sorted array use two
# pointers in O(1) space.
def two_sum_sorted(nums, target):
    # sorted input: two pointers, O(1) extra space
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        total = nums[lo] + nums[hi]
        if total == target:
            return lo, hi
        if total < target:
            lo += 1
        else:
            hi -= 1
    return None


# FOLLOW-UP 2: Only the existence of a subset with the
# target sum.
# Answer: The same subset-sum table with booleans (or a
# bitset: reach |= reach << x) answers existence in O(n *
# sum / word size).
def subset_exists(nums, goal):
    # follow-up: only whether a subset reaches the sum.
    # The
    # set of reachable sums is one integer used as a
    # bitset.
    reach = 1
```

```
    for x in nums:
        reach |= reach << x
    return bool(reach >> goal & 1)


# FOLLOW-UP 3: Negative numbers or a huge sum.
# Answer: With negatives shift all values to be non-
# negative, or use a dictionary from reachable sum to
# count (memoised DFS on (index, sum)); with a huge sum
# the dictionary is only as large as the number of
```

```
# distinct reachable sums.
def count_signs_dict(nums, target):
    # follow-up: negative numbers or a huge sum. A
    # dictionary
    # from "reachable sum" to "number of ways" holds only
    # the
    # sums that really occur.
    seen = Counter({0: 1})
    for x in nums:
        nxt = Counter()
        for s, ways in seen.items():
            nxt[s + x] += ways
            nxt[s - x] += ways
        seen = nxt
    return seen[target]


if __name__ == "__main__":
    assert count_signs([1, 1, 1, 1, 1], 3) == 5
    assert count_signs([1, 2, 1], 2) == 2
    assert count_signs([1, 2, 7, 1, 5, 3], 8) == -1
    assert count_signs([1], 5) == -1
    assert count_signs([0, 0, 1], 1) == 4
    assert count_signs([], 0) == 1
    assert count_signs_dict([1, 1, 1, 1, 1], 3) == 5
    # negative numbers work with the dictionary version
    assert count_signs_dict([-1, 2, 1], 2) == 2
    assert subset_exists([3, 34, 4, 12, 5, 2], 9)
    assert not subset_exists([3, 34, 4], 8)
    assert two_sum([2, 7, 11, 15], 9) == (0, 1)
    assert two_sum([1, 2], 7) is None
    assert two_sum_sorted([1, 3, 4, 8], 11) == (1, 3)
    import random
    for _ in range(300):
        nums = [random.randint(0, 4)
                for _ in range(random.randint(0, 7))]
        target = random.randint(-8, 12)
        want = count_signs_dict(nums, target)
        got = count_signs(nums, target)
        assert got == want or (got == -1 and want == 0)
```

OFFICIAL PAGE TEXT

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples

OFFICIAL PAGE TEXT

```
nums = [1, 1, 1, 1, 1]
targetSum = 3
output: 5

nums = [1, 2, 1]
targetSum = 2
output: 2

nums = [1, 2, 7, 1, 5, 3]
targetSum = 8
output: -1 (Explanation: Since the total sum of the nu
mbers plus the target is odd, it's not possible to rea
ch the target sum.)
```

The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\text{nums} = [a_1, a_2, ..., a_n]nums=[a1,a2,...,an], we need to assign either a +++ or a --- to each number so that the total sum equals a target SSS.Let's denote the sum of the entire list as sum(nums)=totalSum\text{sum(nums)} = \text{totalSum}sum(nums)=totalSum.The problem can be rephrased as finding subsets of nums\text{nums}nums such that the difference between the sum of elements in the subset and the sum of elements not in the subset equals SSS.Formulating the Equation: Let PPP be the sum of elements with a +++ sign and NNN be the sum of elements with a --- sign. We need P-N=SP - N = SP-N=S. We also know that P+N=totalSumP + N = \text{totalSum}P+N=totalSum, where totalSum\text{totalSum}totalSum is the sum of all

OFFICIAL PAGE TEXT

elements in nums\text{nums}nums. Combining the Equations: Adding these two equations:P-N+P+N=S+totalSum2P=S+totalSumP - N + P + N = S + \text{totalSum}\\ 2P = S + \text{totalSum} P-N+P+N=S+totalSum2P=S+totalSumTherefore, P=S+totalSum2P = \frac{S + \text{totalSum}}{2}P=2S+totalSum.The problem now reduces to finding subsets of nums\text{nums}nums whose sum equals S+totalSum2\frac{S + \text{totalSum}}{2}2S+totalSum. Approach We define a recursive function find_subset that takes an index, the list of numbers, the target subset sum, and a memoization table (dp). The function recursively explores two possibilities at each step: Include the current number in the subset. Exclude the current number from the subset. The base cases for the recursion are: If the target subset sum (s) is zero, it means we found a valid subset, so we return 1. If we reach the end of the list without finding the subset sum, we return 0. If we have already computed the value for the current state (index and subset sum), we return the stored value from the memoization table to avoid redundant calculations. The changeSigns function calculates the total sum of the numbers and checks if it is possible to partition them to meet the required target sum S. If it is possible, it initializes the memoization table and

calls the find_subset function to compute the number of ways to achieve the target sum. Solution Time Complexity: The time complexity of this solution is O(n x (totalSum + S)) / 2, where n is the number of elements in the input list. This is because each state in the memoization table can be computed in constant time, and there are n x (totalSum + S) / 2 states to fill. Space Complexity: The space complexity of this solution is O(n x (totalSum + S) / 2) due to the memoization table used to store intermediate results. Additionally, the recursive call stack can go up to O(n) in the worst case, but this is dominated by the space required for the memoization table.Watch our mock Flipkart SWE (software engineering) interview. Neamah asks Rohan (Flipkart SWE) the Target Sum leetcode question.No answers contributed by the community yet.Related questionsFind a triplet in an array with a given sum.Given an array, find the two sum.Find quadruplets in an array with a given sum.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start

OFFICIAL PAGE TEXT

learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Recursion and backtracking

Site tags: grid, backtracking, hard

HARD | 1Point3Acres | onsite-coding | 19 of 20

Matrix Flower Placement with House Adjacency

ASKED 2026-05-31 | 142 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 190/228**

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-05-31 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/matrix-flower-placement-with-house-adjacency

QUESTION

EXAMPLES

```
H 0 0 0
H 0 0 0
0 0 0 H
0 0 0 0
```

The task is to fill some 0 cells with F while satisfying both the row/column constraint and each house's adjacency requirement.

APPROACH

Practice N-Queens with row/column sets and a recursive placement order.
Drill grid-neighbor helpers and constraint pruning before writing the full DFS.
Prepare the boolean version first, then generalize to collect all solutions.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether each row and column holds at most one or exactly one flower, whether a house needs at least one or exactly one adjacent flower, whether adjacency is four-way, and whether to return one solution, a boolean, or all.

BRUTE FORCE TO SAY OUT LOUD

Try every subset of empty cells. Exponential.

OPTIMAL IDEA

Row-by-row backtracking like N-Queens: place at most one flower per row, track used columns, and check each house as soon as the row below it is decided (its neighbors are in rows h - 1 to h + 1). Prune at the first failed house. The boolean version stops at the first solution; the all-solutions version continues.

DRY-RUN THESE CASES

No houses, a house with no empty neighbor (impossible), two houses sharing one neighbor, a 1 x 1 grid.

LIKELY FOLLOW-UPS

Exactly-one rule, all solutions, or a faster exact-cover search.

PITFALLS

Checking a house before the next row is decided. Not restoring the cell on backtrack.

HOW TO THINK ABOUT IT

At most one flower per row and column is the N-Queens pattern: decide row by row which column (if any) gets the flower and remember the used columns. The extra rule about houses is local, so check a house as soon as the last row that can affect it is decided, and prune the branch if it fails.

Basis: N-Queens style backtracking (LeetCode 51/52) with extra local constraints. Assumptions stated explicitly: every house must have at least one flower in a 4-neighbour cell; rows and columns hold at most one flower (or exactly one if requested).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    empty = cells_with(grid, 0)
    count = 0
    for k in range(len(empty) + 1):
        for flowers in combinations(empty, k):
            if not one_per_row_and_col(flowers):
                continue
            if houses_covered(grid, flowers):
                count += 1
    return count
```

TIME	SPACE
O(2^cells)	O(cells)

WHY IT IS SLOW

2^cells subsets: exponential. Backtracking row by row with a used-columns set and early house checks prunes almost all of them (still exponential in the worst case).

OPTIMAL SOLUTION · Row-by-row backtracking with used-column set and delayed house checks

TIME	SPACE
exponential worst case, h... exponential worst case, heavily pruned	O(rows + cols)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What changes between 'at most one' and 'exactly one' flower per row and column?

With 'exactly one' the number of flowers is forced (rows must equal columns) and a row cannot be skipped, which prunes much harder, like N-Queens without the diagonal rule. With 'at most one' you also branch on skipping a row. State which reading you are using before writing code.

FOLLOW-UP 2 How would you speed up the search?

Prune earlier by checking each house as soon as its last neighbour row is decided (done above), order houses by how few candidate cells they have (most-constrained first), and keep bitmasks for used columns. For large grids this becomes a constraint satisfaction or exact-cover problem, so a SAT solver or DLX is the next step. **Cost:** the worst case stays exponential; most-constrained-first ordering and early house checks cut the explored states by orders of magnitude on typical grids.

FOLLOW-UP 3 Exactly one flower per row and column.

Add a flag that forbids skipping a row and requires as many columns as rows (otherwise no solution). The pruning is stronger, since every row must find a column.

FOLLOW-UP 4 Count the solutions instead of listing them.

Keep the same recursion but add one to a counter at the leaf instead of copying the grid; with a bitmask of used columns and memoisation on (row, mask, pending-house state) it can become a DP.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Matrix Flower Placement with House
# Adjacency
#
# Examples: H 0 0 0 H 0 0 0 0 0 0 H 0 0 0 0 The task is to
# fill some 0 cells with F while satisfying both the
# row/column constraint and each house's adjacency
# requirement.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes between 'at most one' and 'exactly one'
# flower per row and column?
# 2. How would you speed up the search?
# 3. Exactly one flower per row and column.
# 4. Count the solutions instead of listing them.
#
# INTERVIEW NOTES
# At most one flower per row and column is the N-Queens
# pattern: decide row by row which column (if any) gets
# the flower and remember the used columns. The extra rule
# about houses is local, so check a house as soon as the
# last row that can affect it is decided, and prune the
# branch if it fails.
#
# BRUTE FORCE (say it first): Choose any subset of empty
# cells and check all the rules
# Enumerate every subset of empty cells, keep those with
# at most one flower per row and column whose every house
# has a flower neighbour, and count them. Exact, the
# oracle, and exponential in the number of cells. Time
# O(2^cells). Space O(cells).


import itertools


# BRUTE FORCE (say it first): Choose any subset of empty
# cells and check all the rules | time O(2^cells)
def count_placements_brute(grid, exactly_one=False):
    R, C = len(grid), len(grid[0])
    empties = [
        (r, c)
        for r in range(R)
        for c in range(C)
        if grid[r][c] == "0"
    ]
    total = 0
    for k in range(len(empties) + 1):
        for chosen in itertools.combinations(
            empties, k
        ):
            rows, cols = [r for r, _ in chosen], [
                c for _, c in chosen
            ]
            if len(set(rows)) != len(rows) or len(
                set(cols)
            ) != len(cols):
                continue
            if exactly_one and not (
                len(rows) == R == C
            ):
                continue
            fl = set(chosen)
            if all(
                any(
                    (r + dr, c + dc) in fl
                    for dr, dc in (
                        (1, 0),
                        (-1, 0),
                        (0, 1),
                        (0, -1),
                    )
                )
                for r in range(R)
                for c in range(C)
                if grid[r][c] == "H"
            ):
                total += 1
    return total


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Row-by-row backtracking with
# used-column set and delayed house checks
# Idea: Place at most one flower per row, tracking used
# columns, and only on empty cells ('0').
def place_flowers(grid, exactly_one=False,
                  find_all=False):
    # 'H' house, '0' empty. At most one F per row and
    # column; every house needs an F on a side.
    R, C = len(grid), len(grid[0])
```

```
    cells = [list(row) for row in grid]
    houses = [(r, c) for r in range(R) for c in range(C)
              if cells[r][c] == "H"]
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if 0 <= a < R and 0 <= b < C \
                    and cells[a][b] == "F":
                return True
        return False

    def row_houses_ok(r):
        return all(house_ok(hr, hc)
                   for hr, hc in houses if hr == r)

    def solve(r, used):
        # FOLLOW-UP 4: find_all=True lists every solution
        if found and not find_all:
            return
        # rows up to r-1 are decided, so row r-2 is final
        if r >= 2 and not row_houses_ok(r - 2):
            return
        if r == R:
            # FOLLOW-UP 3: exactly_one=True: a flower in
            # every column
            full = len(used) == C
            if row_houses_ok(R - 1) and (full or
                                         not exactly_one):
                found.append(["".join(x) for x in cells])
            return
        # try a flower in every free column of this row
        for c in range(C):
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                solve(r + 1, used | {c})
                cells[r][c] = "0"
        # or leave the row empty (not allowed for exactly
        # one)
        if not exactly_one:
```

```
            solve(r + 1, used)

    solve(0, frozenset())
    if find_all:
        return found
    return found[0] if found else None


# FOLLOW-UP 4: Count the solutions instead of listing
# them.
# Answer: Keep the same recursion but add one to a counter
# at the leaf instead of copying the grid; with a bitmask
# of used columns and memoisation on (row, mask, pending-
# house state) it can become a DP.
def count_solutions(grid, exactly_one=False):
    return len(place_flowers(grid, exactly_one, True))


if __name__ == "__main__":
    grid = ["H00", "0H0", "00H"]
    sol = place_flowers(grid)
    assert sol is not None
    # every house has a flower beside it
    for r in range(3):
        for c in range(3):
            if grid[r][c] == "H":
                near = []
                for a, b in STEPS:
                    x, y = r + a, c + b
                    if 0 <= x < 3 and 0 <= y < 3:
```

```
                        near.append(sol[x][y])
                assert "F" in near
    # one flower per row and column
    assert all(row.count("F") <= 1 for row in sol)
    for c in range(3):
        assert [row[c] for row in sol].count("F") <= 1
    # a lone house in a 1 x 1 grid cannot be served
    assert place_flowers(["H"]) is None
    assert count_solutions(["00", "00"], True) == 2
    assert place_flowers(["000", "000"], True) is None
```

NOTES FROM CANDIDATES

This is an N-Queens-style DFS / backtracking problem with extra local constraints around houses.
Track occupied rows and columns, then validate the four-neighbor requirement either incrementally or at the leaf.
The two variants differ on at most one vs exactly one flower per row/column; state the assumption before coding.

PREPARATION

Practice N-Queens with row/column sets and a recursive placement order.
Drill grid-neighbor helpers and constraint pruning before writing the full DFS.
Prepare the boolean version first, then generalize to collect all solutions.

STUDY HERE: topic 6, Strings, Parsing and Recursion | main topic: Parsing

EASY | PracHub | Technical Screen | 20 of 20

Implement string formatting and request tracking tasks

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 159/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Parsing

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: LC 246 Strobogrammatic Number (+ formatting and tracking subtasks) (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-05-21)

Original: https://prachub.com/interview-questions/implement-string-formatting-and-request-tracking-tasks

QUESTION

Overview: This multi-part question evaluates string manipulation, numeric formatting, character-mapping validation under 180° rotation, and event-tracking/order management competencies, assessing parsing, edge-case handling, mapping validation, and design of data structures for ordered output.

APPROACH

Multi-part: numeric/string formatting, validating a number that reads the same after a 180 degree rotation (digit map 0-0, 1-1, 6-9, 8-8, 9-6, two-pointer check), and an ordered request tracker (dict + insertion order). Solve each part independently.

INTERVIEW-DAY PLAN

CLARIFY FIRST

This is four small tasks, so confirm each contract. For the rotation task ask what the rotation map contains and whether leading zeros are allowed. For the tracker ask what 'print in order' means when a later request can still have an earlier end time.

BRUTE FORCE TO SAY OUT LOUD

For each part write the direct version first: split and join, loop with commas, check both readings, and print immediately.

OPTIMAL IDEA

Snake to camel: split on underscores, drop empty parts, capitalize all but the first. Thousands separators: format the absolute value in groups of three from the right and restore the sign. OCR check: validate as-is and after reversing and mapping each character, reject unmapped characters. Tracker: a map for the first timestamp, a min-heap of completed (end, id) and a drain with a watermark.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

snake: leading, trailing and repeated underscores. Commas: 0, negative, 64-bit minimum. OCR: unmapped letter, value 201, value 0. Tracker: end before begin, equal end times.

LIKELY FOLLOW-UPS

Locale-aware formatting, rotation maps given at runtime, and bounded memory in the tracker.

PITFALLS

Negating the minimum 64-bit integer. Printing a request before it is safe to order.

HOW TO THINK ABOUT IT

Four independent tasks, so solve and test each separately and say the contract for each before coding: what an empty or negative input returns, what 'valid' means for the rotated number, and for the tracker when it is safe to print in order. For the tracker, completion is the second event of a pair, and order printing is a heap that is drained only below a watermark.

Basis: Four small tasks: snake_case to camelCase, thousands separators, a rotated-OCR check (LeetCode 246 / 1056 flavor), and a request begin/end tracker that prints in end-time order.

BRUTE FORCE · Task A by walking characters

BRUTE FORCE · PSEUDO-CODE

```
def to_camel(s):
    result, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            result += ch.upper() if up else ch
            up = False
    return result
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear and fine; the split-based version is shorter. The only care is leading and repeated underscores.

BRUTE FORCE · Task B with the language's format specifier

BRUTE FORCE · PSEUDO-CODE

```
def format_number(n):
    return format(n, ",")  # built-in separator
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

BRUTE FORCE · Task B with the language's format specifier

Correct and O(d), but it hides the algorithm; interviews usually ask for the grouping by hand: peel three digits at a time from the right.

BRUTE FORCE · Task C by checking all three readings with a straightforward helper

BRUTE FORCE · PSEUDO-CODE

```
def is_valid_rotation(text):
    if in_range(text):  # a number 1..200
        return True
    if not all(ch in ROTATE for ch in text):
        return False
    turned = [ROTATE[c] for c in reversed(text)]
    return in_range("".join(turned))
```

TIME	SPACE
O(L)	O(L)

WHY IT IS SLOW

It is already linear; there is little to speed up. The risk is correctness: unmapped characters make the rotation invalid, and leading zeros are not valid numbers.

BRUTE FORCE · Task D by storing all events and sorting when asked

BRUTE FORCE · PSEUDO-CODE

```
begin, end = {}, {}
def on_begin(rid, t):
    begin[rid] = t
def on_end(rid, t):
    end[rid] = t
def print_finished():
    done = [r for r in end if r in begin]
    done.sort(key=lambda r: (end[r], r))
    for r in done:
        print(r, begin[r], end[r])
        del begin[r], end[r]
```

TIME	SPACE
O(n log n) per print	O(n)

WHY IT IS SLOW

Each print sorts everything pending, O(n log n); a min-heap of completed requests prints in order with O(log n) per event, releasing only entries below a safe watermark.

OPTIMAL SOLUTION · Task A: snake_case to camelCase

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task B: thousands separators

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task C: valid as-is or after a 180 degree rotation

TIME	SPACE
O(L)	O(L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task D: request tracker that prints completed requests in end-time order

TIME	SPACE
O(log n) per event	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not print as soon as a request completes in the follow-up?

Because another request that completes later can still have an earlier end time. An ordered printer needs a contract for when it is safe to emit: either a watermark (everything with end <= W is final because ends arrive within some known lateness), or a flush at the end of input. State that contract, then the heap pops only below the watermark.

FOLLOW-UP 2 How do you handle bad input (duplicate begin, end before begin, never-completed requests)?

Duplicate begins overwrite or are rejected by policy; an end timestamp earlier than its begin is flagged as invalid rather than printed. Requests that stay in the pending map are the leak to watch: evict by age with a timer or a second heap keyed on first-seen time, and report them as incomplete.

FOLLOW-UP 3 Rotation map provided at runtime with extra pairs.

Nothing changes: the map is a parameter. Validate that it is consistent (rot of rot is the original) if you control it, and treat characters that are not in it as invalid.

FOLLOW-UP 4 The tracker must bound its memory.

Evict requests whose begin or end has been waiting longer than a retention window (a second heap keyed by first-seen time) and report them as incomplete. **Cost:** O(log n) per event for the eviction heap; memory O(requests inside the retention window).

FOLLOW-UP 5 Another locale: a different separator or grouping (Indian style 12,34,567).

Parameterise the separator and the group size: the lowest group may differ from the others (3 digits, then groups of 2 in the Indian system). The loop peels a group from the right each time.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement string formatting and request
# tracking tasks
#
# This multi-part question evaluates string manipulation,
# numeric formatting, character-mapping validation under
# 180° rotation, and event-tracking/order management
# competencies, assessing parsing, edge-case handling,
# mapping validation, and design of data structures for
# ordered output.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not print as soon as a request completes in the
# follow-up?
# 2. How do you handle bad input (duplicate begin, end
# before begin, never-completed requests)?
# 3. Rotation map provided at runtime with extra pairs.
# 4. The tracker must bound its memory.
# 5. Another locale: a different separator or grouping
# (Indian style 12,34,567).
#
# INTERVIEW NOTES
# Four independent tasks, so solve and test each
# separately and say the contract for each before coding:
# what an empty or negative input returns, what 'valid'
# means for the rotated number, and for the tracker when
# it is safe to print in order. For the tracker,
# completion is the second event of a pair, and order
# printing is a heap that is drained only below a
# watermark.
#
# BRUTE FORCE (say it first): Task A by walking characters
# Walk the identifier, set a flag after each underscore,
# and uppercase the next letter. It ignores leading
# underscores by construction when the flag is applied to
# the first output letter, so handle that case explicitly.
# Time O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Task A by walking characters
# | time O(n)
def snake_to_camel_walk(s):
    out, up, started = [], False, False
    for ch in s:
        if ch == "_":
            up = started
        else:
            out.append(ch.upper() if up else ch)
            up, started = False, True
    return "".join(out)


# HELPERS for the base question
ROTATE = {"0": "0", "1": "1", "6": "9",
          "8": "8", "9": "6"}


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def snake_to_camel(s):
    # foo_bar -> fooBar (empty parts are ignored)
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    rest = [w.capitalize() for w in words[1:]]
    return words[0] + "".join(rest)


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
# FOLLOW-UP 5: Another locale: a different separator or
# grouping (Indian style 12,34,567).
# Answer: Parameterise the separator and the group size:
# the lowest group may differ from the others (3 digits,
# then groups of 2 in the Indian system). The loop peels a
# group from the right each time.
def format_number(n, sep=",", sizes=(3,)):
    # FOLLOW-UP 5: sep= and sizes=(3, 2) for other locales
    # group the digits from the right: 1234567 ->
    # 1,234,567.
    # sizes = (3, 2) is the Indian style 12,34,567: the
    # first
    # group has 3 digits, every later one has 2.
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts, i = [], 0
    while len(digits) > sizes[min(i, len(sizes) - 1)]:
        size = sizes[min(i, len(sizes) - 1)]
        # peel one group off the right end
        parts.append(digits[-size:])
        digits = digits[:-size]
        i += 1
    parts.append(digits)
    return sign + sep.join(reversed(parts))


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def is_rotation_valid(s, mapping=ROTATE):
    # does the number read the same upside down (180
    # degrees)?
    # a character not in the map makes it invalid
    if any(ch not in mapping for ch in s):
        return False
    # rotate every digit and read the result backwards
    turned = "".join(mapping[ch] for ch in reversed(s))
    return turned == s
```

```python
# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
class RequestTracker:
    """begin(id, t) and end(id, t). Finished requests are
    printed in the order they BEGAN: one is printed only
    when everything that began before it is done."""

    def __init__(self):
        self.order = deque()
        self.begin_at, self.end_at = {}, {}

    def begin(self, rid, t):
```

```python
        if rid in self.begin_at:
            raise ValueError("duplicate begin")
        self.begin_at[rid] = t
        self.order.append(rid)

    def end(self, rid, t):
        if rid not in self.begin_at:
            raise ValueError("end without begin")
        if t < self.begin_at[rid]:
            raise ValueError("ends before it begins")
        self.end_at[rid] = t
        # print from the front while the oldest are
        # finished
        out = []
        while self.order and self.order[0] in self.end_at:
            first = self.order.popleft()
            out.append((first, self.begin_at[first],
                        self.end_at[first]))
        return out


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("__a__b_") == "aB"
    assert snake_to_camel("") == ""
    assert format_number(1234567) == "1,234,567"
    assert format_number(-1000) == "-1,000"
    assert format_number(999) == "999"
    assert format_number(0) == "0"
    indian = format_number(1234567, sizes=(3, 2))
    assert indian == "12,34,567"
    assert format_number(1234, ".") == "1.234"
    assert is_rotation_valid("69")
    assert is_rotation_valid("818")
    assert not is_rotation_valid("962")
    assert not is_rotation_valid("12")
    assert is_rotation_valid("") is True
    t = RequestTracker()
    t.begin("a", 1)
    t.begin("b", 2)
    # b finished first, but a has not: nothing can be
    # printed
    assert t.end("b", 5) == []
    # a finishes: both are printed, in begin order
    assert t.end("a", 7) == [("a", 1, 7), ("b", 2, 5)]
```

```python
    again = lambda: t.begin("a", 9)
    orphan = lambda: t.end("zz", 1)
    for call in (again, orphan):
        try:
            call()
            raise AssertionError("should fail")
        except ValueError:
            pass
```

FULL QUESTION TEXT FROM THE SITE

You are given several small coding tasks. Implement each as described.

Task A — Convert snake_case to camelCase

Write a function that converts an identifier from snake_case to camelCase.

Input: a string s containing lowercase letters a-z and underscores _ .
Output: the camelCase version.
Rules/edge cases:
Consecutive underscores should be treated as a single word separator (ignore empty parts).
If s starts/ends with underscores, ignore those underscores.
The first word remains lowercase; each subsequent word is capitalized.

Example:

"hello_world" -> "helloWorld"
"__my__var__name_" -> "myVarName"
Task B — Format an integer with thousands separators

Write a function that formats a (possibly negative) integer with commas as thousands separators.

Input: an integer n (may be large; assume it fits in 64-bit signed integer).
Output: a string representation using commas every 3 digits from the right.

Examples:

123 -> "123"
1234 -> "1,234"
-12345678 -> "-12,345,678"
Task C — Validate an OCR string under 180° rotation

An OCR system outputs a string t that may contain digits and a few letters. You need to decide whether it can represent an integer between 1 and 200 inclusive under either normal reading or after a 180-degree rotation.

Define a rotation mapping rot(c) for characters; rotation is valid only if every character in the string has a mapping. Rotating a whole string means:

reverse the string, and
replace each character c with rot(c) .

Return true if either:

t as-is is a valid base-10 integer in [1, 200] , OR
rotate180(t) is a valid base-10 integer in [1, 200] .

Assume the rotation map is provided as a dictionary (examples from the interview note included):

'6' <-> '9'
plus any additional pairs given at runtime (e.g., 'h' -> '4' , 'L' -> '7'). Characters not in the map are invalid for rotation.

Examples (assuming map includes 6<->9, 0->0, 1->1, 8->8):

FULL QUESTION TEXT FROM THE SITE

"69" rotates to "69" and is valid (69 in range) -> true
"201" is out of range, rotate may or may not help -> depends on mapping -> compute
Task D — Track request begin/end times and print in a required order

You are implementing a Server class that receives events for requests. Each request has a unique requestId.

You must implement two methods:

begin(requestId, timestamp)
end(requestId, timestamp)
Part 1

Whenever both timestamps for a request are known, print/log a line:

requestId begin=<beginTs> end=<endTs>

Assumptions:

Events may arrive out of order (an end can arrive before begin). There may be multiple in-flight requests.
Follow-up

Instead of printing immediately when a pair is complete, the server must always print completed requests in increasing order of end time (earliest end printed first). If multiple have the same end time, break ties by requestId.

Design the data structures and logic to support this ordering efficiently.